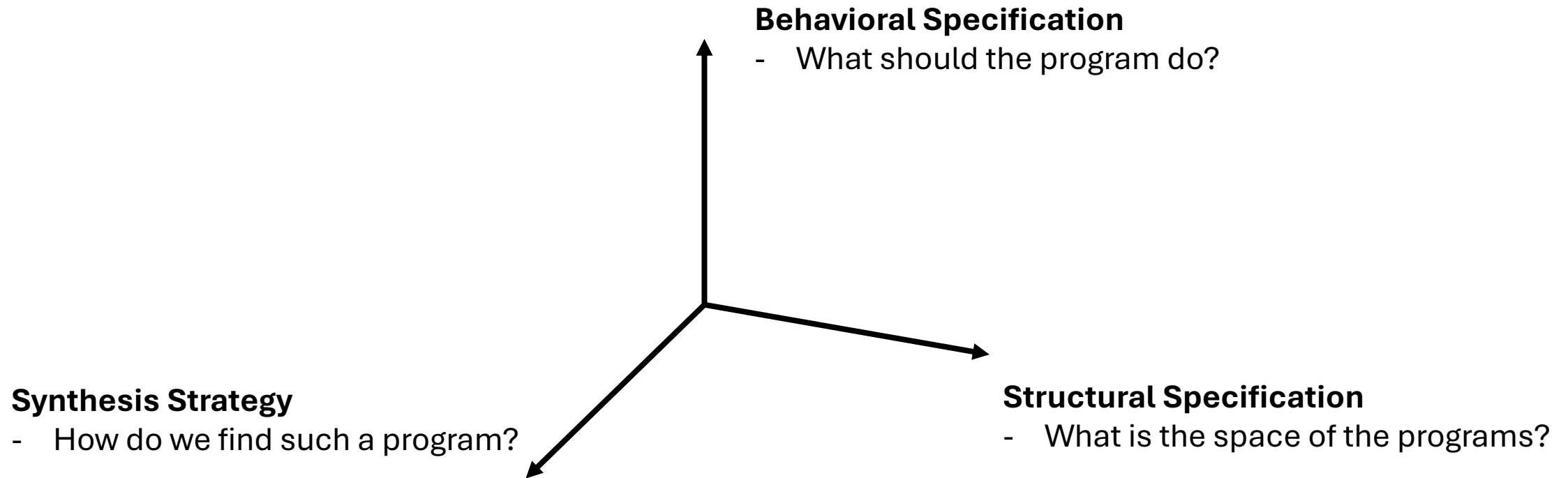


Machine Programming

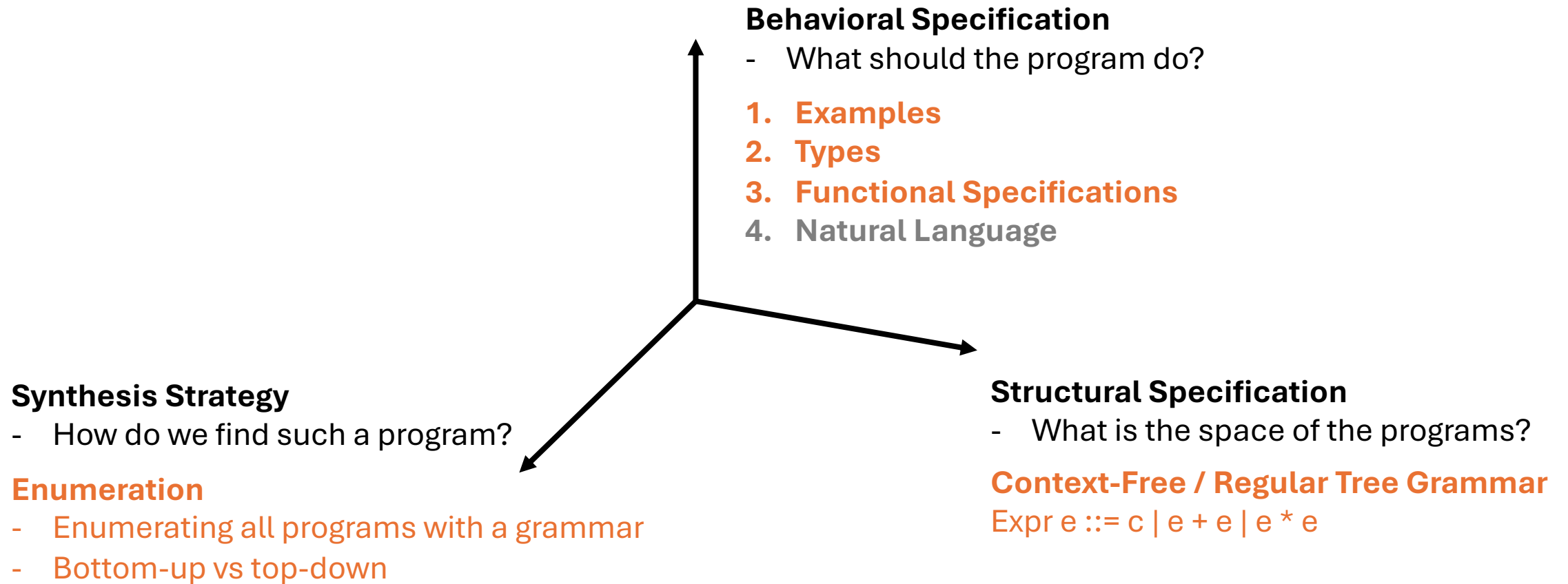
Lecture 5 – Language Modeling for Synthesis

Ziyang Li

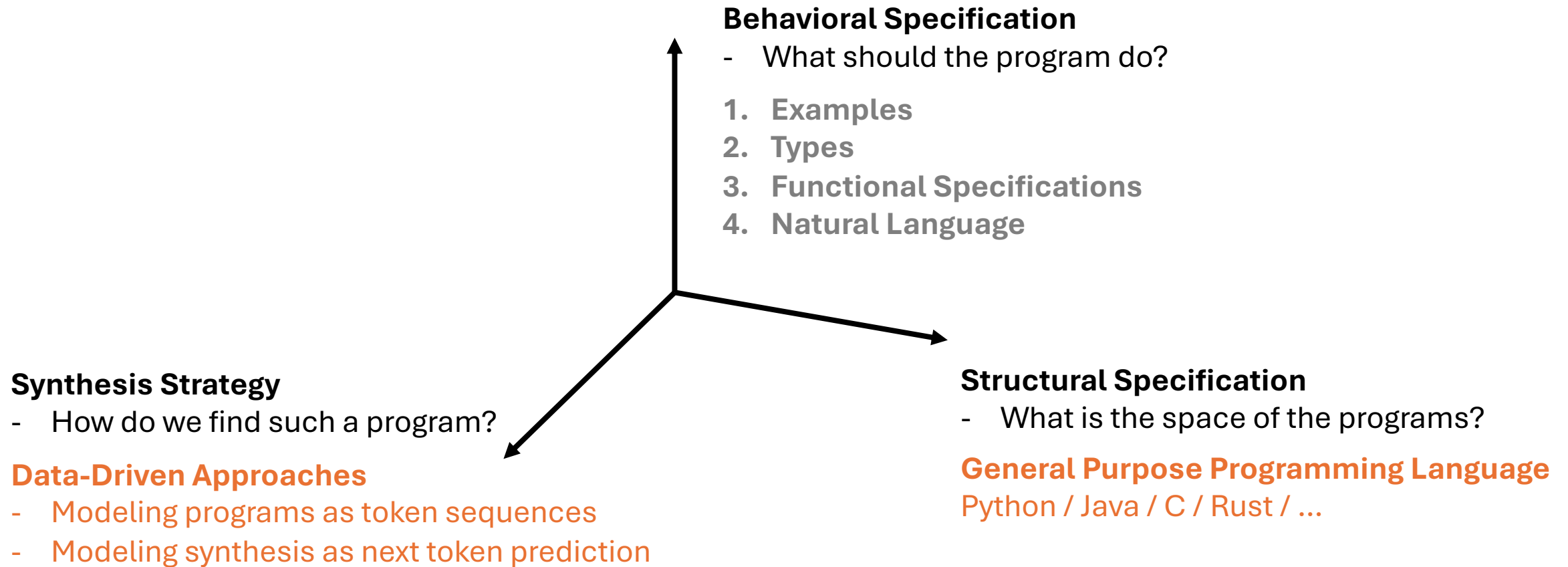
Dimensions in Program Synthesis



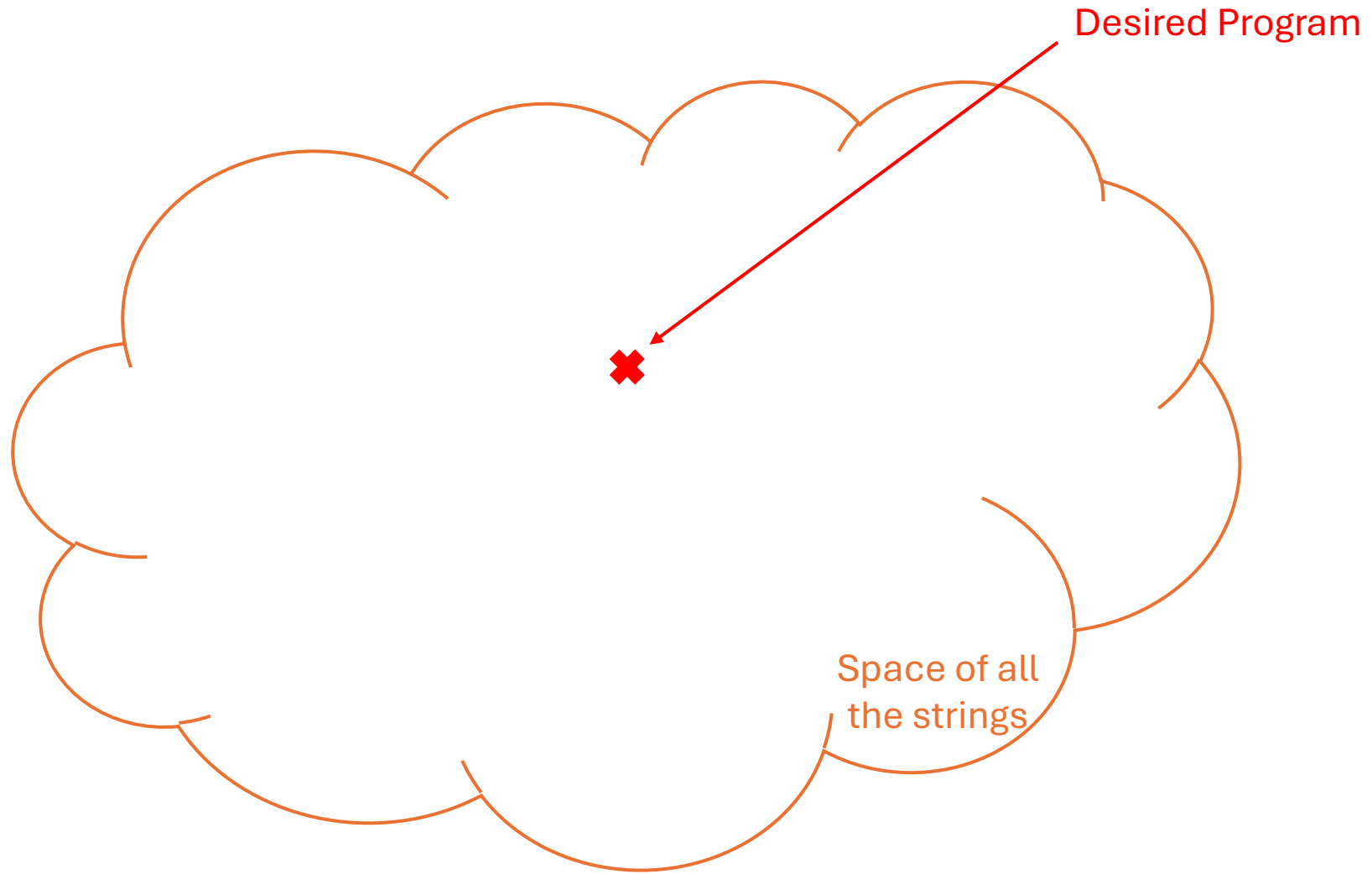
The Course So Far



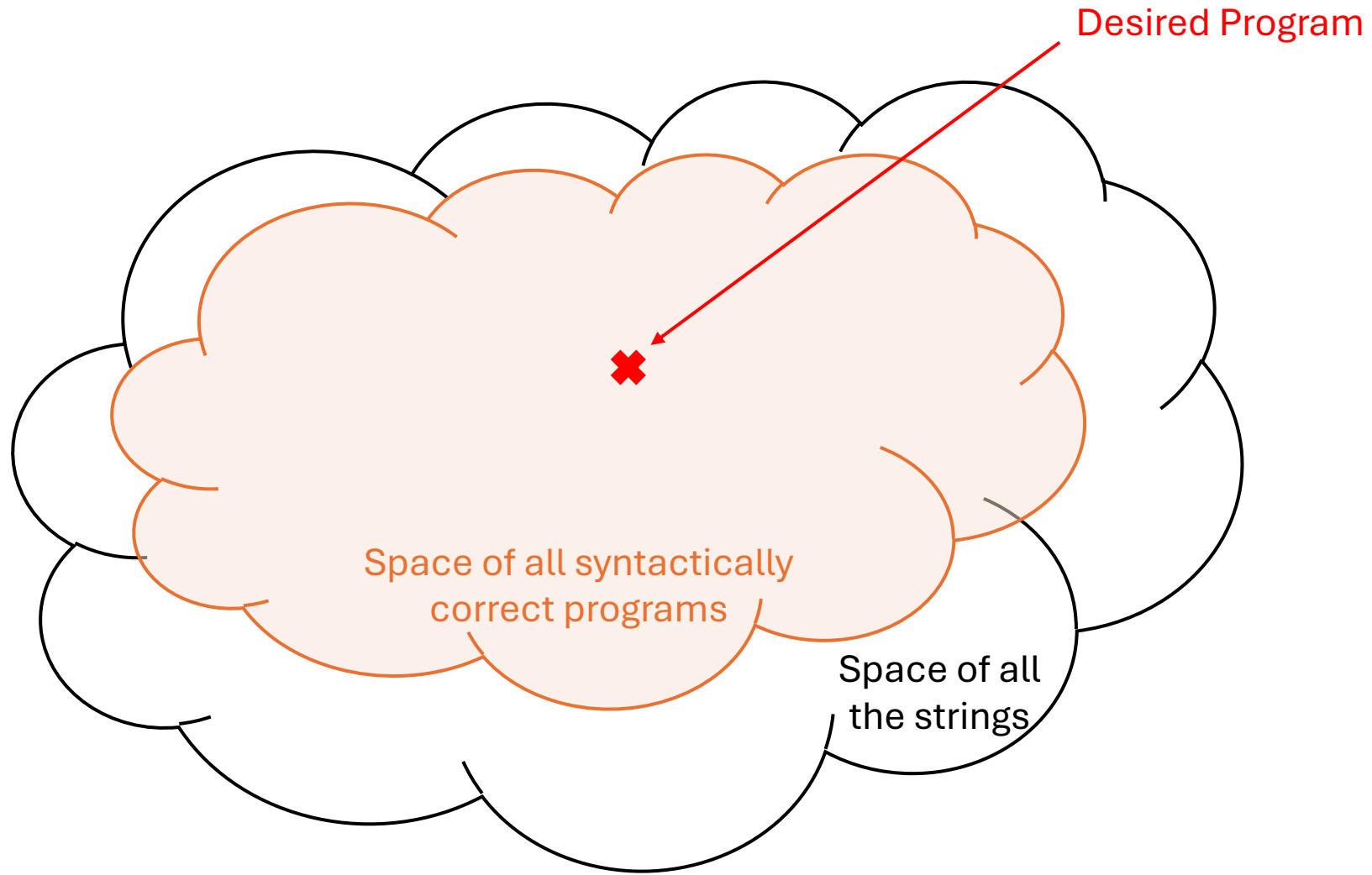
Today



High Level Picture

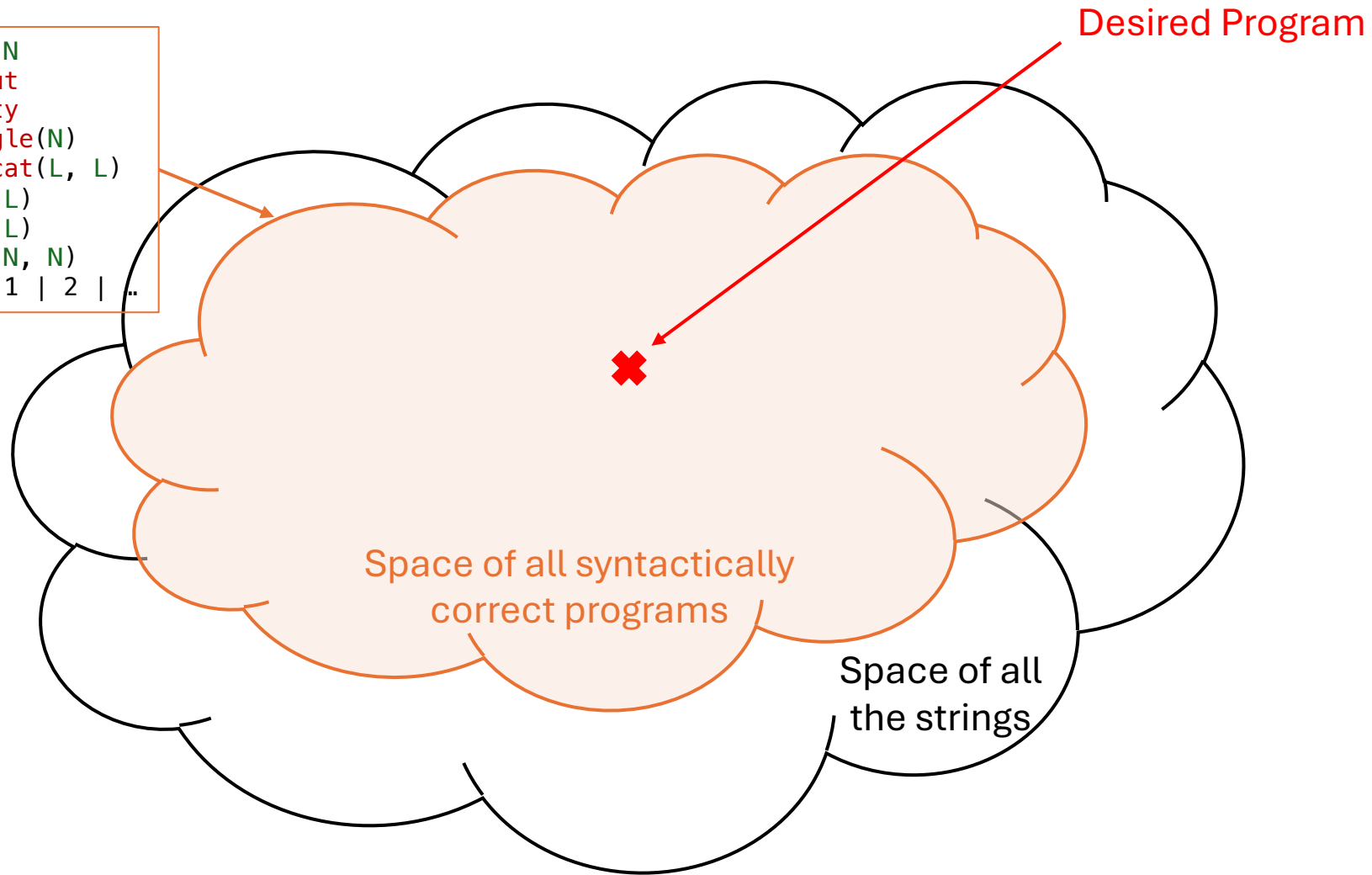


High Level Picture



High Level Picture

```
(Program) P ::= L | N
(List) L ::= input
           | empty
           | single(N)
           | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ..
```



```
(Program) P ::= L | N
(List) L ::= input
          | empty
          | single(N)
          | concat(L, L)
(Number) N ::= len(L)
            | min(L)
            | add(N, N)
            | 0 | 1 | 2 | ...
```

Small Domain Specific Language (DSL)

```

stmt = FunctionDef(identifier name, arguments args,
                    stmt* body, expr* decorator_list, expr? returns,
                    string? type_comment, type_param* type_params)
| AsyncFunctionDef(identifier name, arguments args,
                   stmt* body, expr* decorator_list, expr? return,
                   string? type_comment, type_param* type_params)

| ClassDef(identifier name,
           expr* bases,
           keyword* keywords,
           stmt* body,
           expr* decorator_list,
           type_param* type_params)
| Return(expr? value)

| Delete(expr* targets)
| Assign(expr* targets, expr value, string? type_comment)
| TypeAlias(expr name, type_param* type_params, expr value)
| AugAssign(expr target, operator op, expr value)
-- 'simple' indicates that we annotate simple name without pare
| AnnAssign(expr target, expr annotation, expr? value, int simp

-- use 'orelse' because else is a keyword in target languages
| For(expr target, expr iter, stmt* body, stmt* orelse, string?
| AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, st
| While(expr test, stmt* body, stmt* orelse)
| If(expr test, stmt* body, stmt* orelse)
| With(withitem* items, stmt* body, string? type_comment)
| AsyncWith(withitem* items, stmt* body, string? type_comment)

| Match(expr subject, match_case* cases)

| Raise(expr? exc, expr? cause)
| Try(stmt* body, except_handler* handlers, stmt* orelse, stmt*
| TryStar(stmt* body, except_handler* handlers, stmt* orelse, st
| Assert(expr test, expr? msg)

| Import(alias* names)
| ImportFrom(identifier? module, alias* names, int? level)

| Global(identifier* names)
| Nonlocal(identifier* names)
| Expr(expr value)
| Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
| NamedExpr(expr target, expr value)
| BinOp(expr left, operator op, expr right)
| UnaryOp(unaryop op, expr operand)
| Lambda(arguments args, expr body)
| IfExp(expr test, expr body, expr orelse)
| Dict(expr* keys, expr* values)
| Set(expr* elts)
| ListComp(expr elt, comprehension* generators)
| SetComp(expr elt, comprehension* generators)
| DictComp(expr key, expr value, comprehension* generators)
| GeneratorExp(expr elt, comprehension* generators)
-- the grammar constrains where yield expressions can occur
| Await(expr value)
| Yield(expr? value)
| YieldFrom(expr value)
-- need sequences for compare to distinguish between
-- x < 4 < 3 and (x < 4) < 3
| Compare(expr left, cmpop* ops, expr* comparators)
| Call(expr func, expr* args, keyword* keywords)
| FormattedValue(expr value, int conversion, expr? format_spec)
| JoinedStr(expr* values)
| Constant(constant value, string? kind)

-- the following expression can appear in assignment context
| Attribute(expr value, identifier attr, expr_context ctx)
| Subscript(expr value, expr slice, expr_context ctx)
| Starred(expr value, expr_context ctx)
| Name(identifier id, expr_context ctx)
| List(expr* elts, expr_context ctx)
| Tuple(expr* elts, expr_context ctx)

-- can appear only in Subscript
| Slice(expr? lower, expr? upper, expr? step)

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

expr_context = Load | Store | Del

boolop = And | Or

operator = Add | Sub | Mult | MatMult | Div | Mod | Pow | LShift
          | RShift | BitOr | BitXor | BitAnd | FloorDiv

unaryop = Invert | Not | UAdd | USub

cmpop = Eq | NotEq | Lt | LtE | Gt | GtE | Is | IsNot | In | NotIn

comprehension = (expr target, expr iter, expr* ifs, int is_async)

```

Formal Python 3 Abstract Syntax

<https://docs.python.org/3/library/ast.html>

```
    -- BoolOp() can use left & right?  
expr = BoolOp(boolop op, expr* values)  
    | NamedExpr(expr target, expr value)  
    | BinOp(expr left, operator op, expr right)  
    | UnaryOp(unaryop op, expr operand)  
    | Lambda(arguments args, expr body)  
    | IfExp(expr test, expr body, expr orelse)  
    | Dict(expr* keys, expr* values)  
    | Set(expr* elts)  
    | ListComp(expr elt, comprehension* generators)
```

```
-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
      | NamedExpr(expr target, expr value)
      | BinOp(expr left, operator op, expr right)
      | UnaryOp(unaryop op, expr operand)
      | Lambda(arguments args, expr body)
      | IfExp(expr test, expr body, expr orelse)
      | Dict(expr* keys, expr* values)
      | Set(expr* elts)
      | ListComp(expr elt, comprehension* generators)
```

```
lambda a, b: a < b
```

```
Expr(
  value=Lambda(
    args=arguments(
      args=[
        arg(arg='a'),
        arg(arg='b')]
      ),
    body=Compare(
      left=Name(id='a'),
      ops=[Lt()],
      comparators=[
        Name(id='b')
      ]
    )
  )
)
```

```
UnaryOp(unaryop op, expr operand)
Lambda(arguments args, expr body)
IfExp(expr test, expr body, expr orelse)
Dict(expr* keys, expr* values)
Set(expr* elts)
ListComp(expr elt, comprehension* generators)
SetComp(expr elt, comprehension* generators)
DictComp(expr key, expr value, comprehension* ge
GeneratorExp(expr elt, comprehension* generators
— the grammar constrains where yield expressions
Await(expr value)
```

```
res = [i for i in arr if i > 0]
```

```
Assign(
  targets=[Name(id='res')],
  value=ListComp(
    elt=Name(id='i'),
    generators=[
      comprehension(
        target=Name(id='i'),
        iter=Name(id='arr'),
        ifs=[
          Compare(
            left=Name(id='i'),
            ops=[Gt()],
            comparators=[
              Constant(value=0)
            ]
          )
        ]
      )
    ]
  )
)
```

write me a python program that prints the ast of the program itself, while computing the depth and the size (number of AST nodes within the program)

Thought for 9s

python

Copy code

```
#!/usr/bin/env python3
"""
Self-analysing script:
- Parses its own source into a Python AST
- Prints the AST (pretty-dumped)
- Computes and prints:
    * size = total number of AST nodes (including the root `Module`)
    * depth = maximum path length from the root to any leaf (in nodes)
"""

import ast
import inspect
import sys

def ast_size(node: ast.AST) -> int:
    """Total count of nodes in the AST (includes the root)."""
    return sum(1 for _ in ast.walk(node))

def ast_depth(node: ast.AST) -> int:
    """Maximum depth of the AST (in nodes). Leaf has depth 1."""
    children = list(ast.iter_child_nodes(node))
    if not children:
        return 1
    return 1 + max(ast_depth(ch) for ch in children)

def main() -> None:
    # Get the current module's source (more robust than reading __file__)
    src = inspect.getsource(sys.modules[__name__])
```

+ Ask anything



Python: 43 lines

```

10 import ast
11 import inspect
12 import sys
13
14 def ast_size(node: ast.AST) -> int:
15     """Total count of nodes in the AST (includes the root)."""
16     return sum(1 for _ in ast.walk(node))
17
18 def ast_depth(node: ast.AST) -> int:
19     """Maximum depth of the AST (in nodes). Leaf has depth 1."""
20     children = list(ast.iter_child_nodes(node))
21     if not children:
22         return 1
23     return 1 + max(ast_depth(ch) for ch in children)
24
25 def main() -> None:
26     """# Get the current module's source (more robust than reading __file__)
27     """
28     src = inspect.getsource(sys.modules[__name__])
29     tree = ast.parse(src)
30
31     """# Pretty-print the AST
32     """
33     print("=== AST (ast.dump with indentation) ===")
34     print(ast.dump(tree, indent=2))
35
36     """# Compute metrics
37     """
38     size = ast_size(tree)
39     depth = ast_depth(tree)
40
41     print("\n=== Metrics ===")
42     print(f"Size (number of AST nodes): {size}")
43     print(f"Depth (max root-to-leaf length): {depth}")
44
45 if __name__ == "__main__":
46     main()

```



AST: 181 Nodes, Depth 10

[illegible]

Python: 43 lines

```
10 import ast
11 import inspect
12 import sys
13
14 def ast_size(node: ast.AST) -> int:
15     """Total count of nodes in the AST (includes the root)."""
16     return sum(1 for _ in ast.walk(node))
17
18 def ast_depth(node: ast.AST) -> int:
19     """Maximum depth of the AST (in nodes). Leaf has depth 1."""
20     children = list(ast.iter_child_nodes(node))
21     if not children:
22         return 1
23     return 1 + max(ast_depth(ch) for ch in children)
24
25 def main() -> None:
26     """# Get the current module's source (more robust than reading __file__)
27     src = inspect.getsource(sys.modules[__name__])
28     tree = ast.parse(src)
29
30     """# Pretty-print the AST
31     print("=== AST (ast.dump with indentation) ===")
32     print(ast.dump(tree, indent=2))
33
34     """# Compute metrics
35     size = ast_size(tree)
36     depth = ast_depth(tree)
37
38     print("\n=== Metrics ===")
39     print(f"Size (number of AST nodes): {size}")
40     print(f"Depth (max root-to-leaf length): {depth}")
41
42 if __name__ == "__main__":
43     main()
```



```
alias(name='sys'))],
FunctionDef(
    name='ast_size',
    args=arguments(
        args=[
            arg(
                arg='node',
                annotation=Attribute(
                    value=Name(id='ast', ctx=Load()),
                    attr='AST',
                    ctx=Load()))]),
    body=[
        Expr(
            value=Constant(value='Total count of nodes
in the AST (includes the root).')),
        Return(
            value=Call(
                func=Name(id='sum', ctx=Load()),
                args=[
                    GeneratorExp(
                        elt=Constant(value=1),
                        generators=[
                            comprehension(
                                target=Name(id='_', ctx=Store()),
                                iter=Call(
                                    func=Attribute(
                                        value=Name(id='ast',
                                                    ctx=Load()),
                                        attr='walk',
                                        ctx=Load()),
                                    args=[
                                        Name(id='node', ctx=Load())],
                                is_async=0)))])),
                returns=Name(id='int', ctx=Load()))],
FunctionDef(
    name='ast_depth',
    args=arguments(
        args=[
            arg(
                arg='node',
                annotation=Attribute(
                    value=Name(id='ast', ctx=Load()),
                    attr='AST',
                    ctx=Load()))]),
    body=[
        Expr(
```

Syntax: How big is the space of programs?

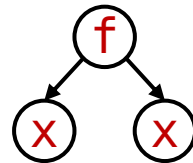
$E ::= x \mid f(E, E)$

Depth ≤ 0



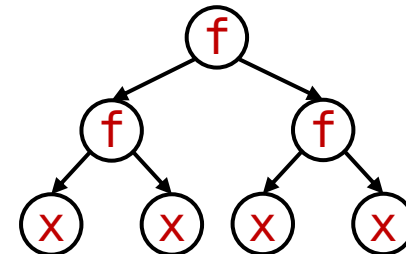
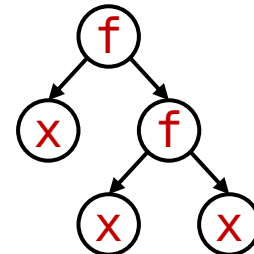
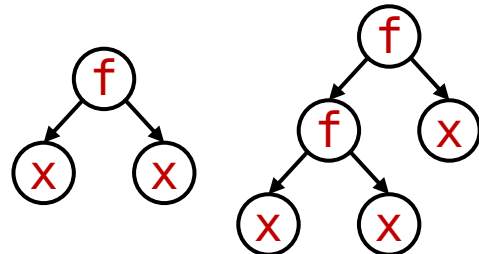
$\text{Size}(0) = 1$

Depth ≤ 1



$\text{Size}(1) = 2$

Depth ≤ 2



$\text{Size}(2) = 5$

$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

Syntax: How big is the space of programs?

$$E ::= x \mid f(E, E)$$
$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

```
size(1) = 1
size(2) = 2
size(3) = 5
size(4) = 26
size(5) = 677
size(6) = 458330
size(7) = 210066388901
size(8) = 44127887745906175987802
size(9) = 1947270476915296449559703445493848930452791205
size(10) = 3791862310265926082868235028027893277370233152247388584761734150717768254410341175325352026
```

Syntax: How big is the space of programs?

$$E ::= x \mid f_1(E, E) \mid \dots \mid f_n(\overbrace{E_1, E_2, \dots, E_m}^{m \text{ arguments}})$$

n production rules
Size(depth) = ???

$$E ::= x \mid f(E, E)$$

$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

Syntax: How big is the space of programs?

$$E ::= x \mid f_1(E, E) \mid \dots \mid f_n(\overbrace{E_1, E_2, \dots, E_m}^{m \text{ arguments}})$$

n production rules

$$\text{Size}(0) = 1$$

$$\text{Size}(\text{depth} + 1) = 1 + (n - 1) \text{Size}(\text{depth})^m$$

$$E ::= x \mid f(E, E)$$

$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

Syntax: How big is the space of programs?

$$E ::= x \mid f_1(E, E) \mid \dots \mid f_n(\overbrace{E_1, E_2, \dots, E_m}^{m \text{ arguments}})$$

n production rules

$$\text{Size}(\emptyset) = 1$$

$$\text{Size}(\text{depth} + 1) = 1 + (n - 1) \text{Size}(\text{depth})^m$$

$$\text{Size}(d) = \mathcal{O}(n^{m^d})$$

$$E ::= x \mid f(E, E)$$

$$\text{Size}(\text{depth}) = 1 + \text{Size}(\text{depth} - 1)^2$$

How big is the space of Python programs?

```
stmt = FunctionDef(identifier name, arguments args,
                    stmt* body, expr* decorator_list, expr? returns,
                    string? type_comment, type_param* type_params)
| AsyncFunctionDef(identifier name, arguments args,
                    stmt* body, expr* decorator_list, expr? returns,
                    string? type_comment, type_param* type_params)

| ClassDef(identifier name,
            expr* bases,
            keywords keywords,
            stmt* body,
            expr* decorator_list,
            type_param* type_params)
| Return(expr? value)

| Delete(expr* targets)
| Assign(expr* targets, expr value, string? type_comment)
| TypeAlias(expr name, type_param* type_params, expr value)
| AugAssign(expr target, operator op, expr value)
-- 'simple' indicates that we annotate simple name without parens
| AnnAssign(expr target, expr annotation, expr? value, int simple)

-- use 'orelse' because else is a keyword in target languages
| For(expr target, expr iter, stmt* body, stmt* orelse, string? type_comment)
| AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, string? type_comment)
| While(expr test, stmt* body, stmt* orelse)
| If(expr test, stmt* body, stmt* orelse)
| With(withitem* items, stmt* body, string? type_comment)
| AsyncWith(withitem* items, stmt* body, string? type_comment)
| Match(expr subject, match_case* cases)

| Raise(expr? exc, expr? cause)
| Try(stmt* body, except_handler* handlers, stmt* orelse, stmt* finalizer)
| TryStar(stmt* body, except_handler* handlers, stmt* orelse, stmt* finalizer)
| Assert(expr test, expr? msg)

| Import(alias* names)
| ImportFrom(identifier? module, alias* names, int? level)

| Global(identifier* names)
| Nonlocal(identifier* names)
| Expr(expr value)
| Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? end_col_offset)

-- BoolOp() can use left & right?
| BoolOp(boolop op, expr* values)
| NamedExpr(expr target, expr value)
| BinOp(expr left, operator op, expr right)
| UnaryOp(unaryop op, expr operand)
| Lambda(arguments args, expr body)
| IfExp(expr test, expr body, expr orelse)
| Dict(expr* keys, expr* values)
| Set(expr* elts)
| ListComp(expr elt, comprehension* generators)
| SetComp(expr elt, comprehension* generators)
| DictComp(expr key, expr value, comprehension* generators)
| GeneratorExp(expr elt, comprehension* generators)

-- the grammar constrains where yield expressions can occur
| Await(expr value)
| Yield(expr? value)
| YieldFrom(expr value)

-- need sequences for compare to distinguish between
-- x < 4 < 3 and (x < 4) < 3
| Compare(expr left, cmpops ops, expr* comparators)
| Call(expr func, expr* args, keywords keywords)
| FormattedValue(expr value, int conversion, expr? format_spec)
| JoinedStr(expr* values)
| Constant(constant value, string? kind)

-- the following expression can appear in assignment context
| Attribute(expr value, identifier attr, expr_context ctx)
| Subscript(expr value, expr slice, expr_context ctx)
| Starred(expr value, expr_context ctx)
| Name(identifier id, expr_context ctx)
| List(expr* elts, expr_context ctx)
| Tuple(expr* elts, expr_context ctx)

-- can appear only in Subscript
| Slice(expr? lower, expr? upper, expr? step)

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? end_col_offset)

expr_context = Load | Store | Del

boolop = And | Or

operator = Add | Sub | Mult | MatMult | Div | Mod | Pow | LShift
          | RShift | BitOr | BitXor | BitAnd | FloorDiv

unaryop = Invert | Not | UAdd | USub

cmpop = Eq | NotEq | Lt | LtE | Gt | GtE | Is | IsNot | In | NotIn

comprehension = (expr target, expr iter, expr* ifs, int is_async)
```

$$\text{Size}(0) = 1$$

$$\text{Size}(\text{depth} + 1) = 1 + (n - 1) \text{Size}(\text{depth})^m$$

$$\text{Size}(d) = O(n^{m^d})$$

Python 3:

- Production rules: ~ 110 ($n \approx 110$)
- Maximum Arity: ~ 7 ($m \approx 7$)

Note:

- Not counting that there are an enormous number of integers and string literals

General Purpose Programming Languages

```

state = FunctionDef(identifier_name, arguments args,
                    stmts=body, expr_decorator_list, expr returns,
                    string type_comment, type_params type_params)
| AsyncFunctionDef(identifier_name, arguments args,
                   stmts=body, expr_decorator_list, expr returns,
                   string type_comment, type_params type_params)

ClassDef(identifier_name,
         expr bases,
         keywords keywords,
         stmts=body,
         expr_decorator_list,
         type_params type_params)
Return(expr? value)

Delete(expr targets)
Assign(expr targets, expr value, string type_comment)
TypeAlias(expr name, type_params type_params, expr value)
AugAssign(expr target, operator op, expr value)
-- stmt indicates that we can annotate single name without para
AnnAssign(expr target, expr annotation, expr? value, int? simp

-- use 'orelse' because else is a keyword in target languages
For(expr target, expr iter, stmts=body, stmts=orelse, string
ExprFor(expr target, expr iter, stmts=body, stmts=orelse, st
While(expr test, stmts=body, stmts=orelse)
If(expr test, stmts=body, stmts=orelse)
WithItem(withitem=expr, stmts=body, string type_comment)
AsyncWith(withitem=items, stmts=body, string type_comment)

Match(expr subject, match_cases cases)

Raise(expr exc, expr? cause)
Try(stmts=body, except_handlers handlers, stmts=orelse, string
TryStar(stmts=body, except_handlers handlers, stmts=orelse, st
Assert(expr test, expr msg)

Import(alias names)
ImportFrom(identifier) module, alias names, int? level)

Global(identifier names)
NonLocal(identifier names)
Expr(expr value)
Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

-- BoolOp() can use left & right?
expr = BoolOp(BoolOp op, expr values)
NeedsExpr(expr target, comprehension generators)
BinOp(expr left, operator op, expr right)
UnaryOp(unaryop op, expr operand)
Lambda(arguments args, expr body)
IfExpr(expr test, expr body, expr? orElse)
IfExpr keywords, exprs=elmts)
Set(exprs elmts)
ListComp(expr elt, comprehension generators)
SetComp(expr elt, comprehension generators)
DictComp(expr key, expr value, comprehension generators)
GeneratorExpr(expr elt, comprehension generators)

-- the grammar constrains where yield expressions can occur
Await(expr value)
Yield(expr value)
YieldFrom(expr value)

-- need sequences for context to distinguish between
-- x < 4 & (x < 4) & x
Compare(expr left, cmpops ops, expr comparators)
ExprComp(func, expr args, keywords keywords)
NamedExpr(value, expr context ctx)
JoinStr(expr values, int? conversion, expr? format_spec)
JoinedStr(expr values)
Constant(constant value, string? kind)

-- the following expression can appear in assignment context
Attribute(expr value, identifier attr, expr context ctx)
Subscript(expr value, expr context ctx, expr context ctx)
Starred(expr value, expr context ctx)
Name(identifier id, expr context ctx)
List(exprs elmts, expr context ctx)
Tuple(exprs elmts, expr context ctx)

-- can appear only in Subscript
| Slice(expr? lower, expr? upper, expr? step)

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

expr_context = Load | Store | Del

boolop = And | Or

operator = Add | Sub | Mult | MatMult | Div | Mod | Pow | LShift
         | RShift | BitOr | BitXor | BitAnd | FloorDiv

unaryop = Invert | Not | UAdd | USub

cmpop = Eq | NotEq | Lt | LE | GE | GtE | Is | IsNot | In | NotIn

comprehension = (expr target, expr iter, exprs ifs, int? is_async)

```

[illegible]

```

grammar Closure;
file_ : form* EOF ;
form : literal | list_ | vector | map_ | reader_macro ;
forms : form* ;
list_ : '(' forms ')';
vector : '[' forms ']';
map_ : '{' (form form)* '}';
set_ : '#{ forms }';
reader_macro : lambda_ | meta_data | regex | var_quote | host_expr | set_
              | tag | discard | dispatch | deref | quote
              | backtick | unquote | unquote_splicing | gensym ;
quote : '\"' form ;
backtick : '`' form ;
unquote : '~' form ;
unquote_splicing : '~@' form ;
tag : '^' form form ;
deref : '@' form ;
gensym : SYMBOL '#' ;
lambda_ : '#' (form* ')';
meta_data : '#^' (map_ form | form) ;
var_quote : '#\"' symbol ;
host_expr : '#+' form form ;
discard : '#-' form ;
dispatch : '#' symbol form ;
regex : '#' string_ ;
literal : string_ | number | character | nil_
         | BOOLEAN | keyword | symbol | param_name ;
string_ : STRING ;
hex_ : HEX ;
bin_ : BIN ;
bign_ : BIGN ;
number : FLOAT
        | hex_ | bin_ | bign_ | LONG ;
character : named_char | u_hex_quad | any_char ;
named_char : CHAR_NAMED ;
any_char : CHAR_ANY ;
u_hex_quad : CHAR_U ;
nil_ : NIL ;
keyword : macro_keyword | simple_keyword ;
simple_keyword : ':' symbol ;
macro_keyword : ':' ':' symbol ;
symbol : ns_symbol | simple_sym ;
simple_sym : SYMBOL ;
ns_symbol : NS_SYMBOL ;
param_name : PARAM_NAME ;

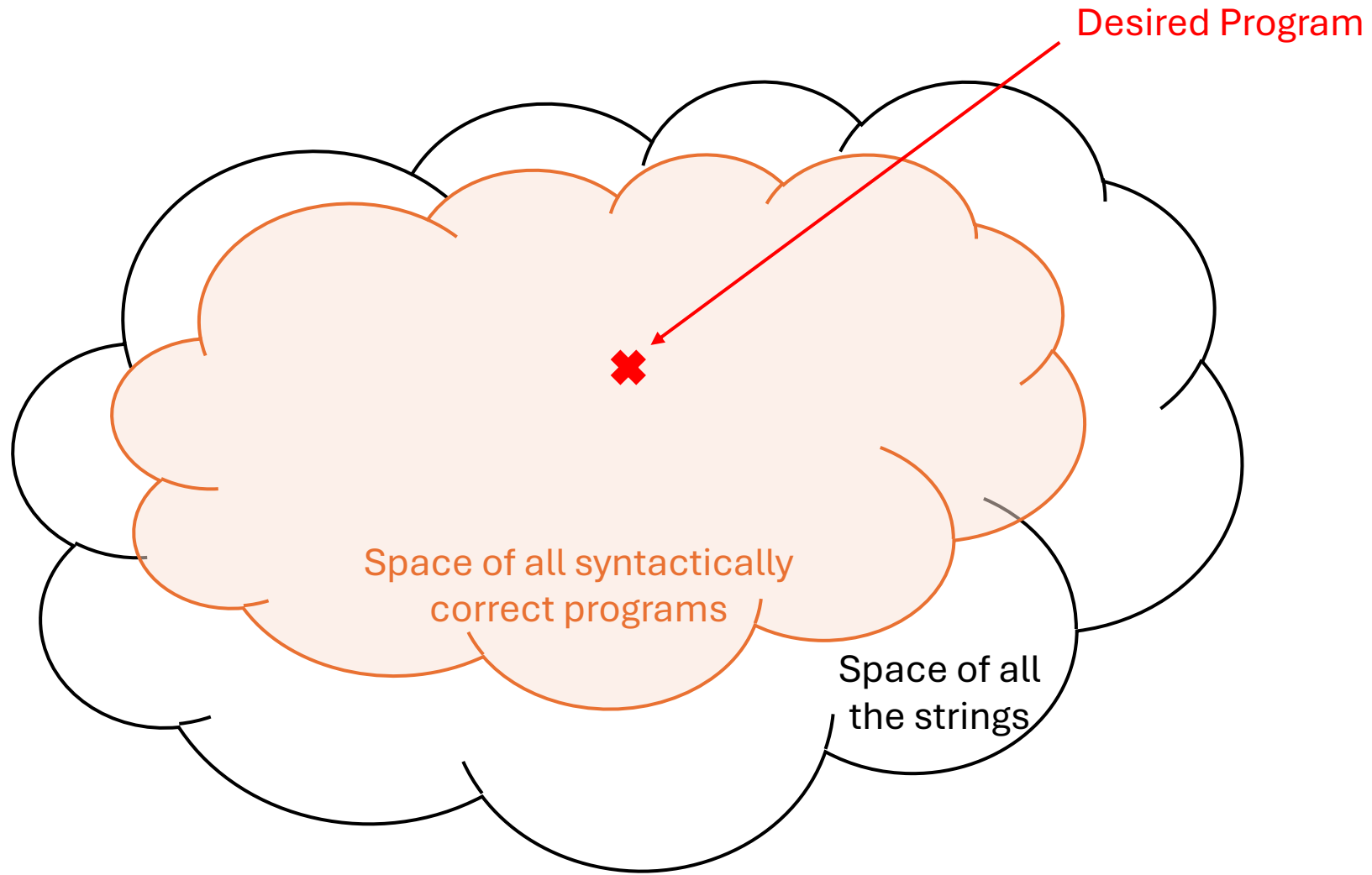
```

Clojure ([link](#))

Python ([link](#))

C (link)

High Level Picture



```
-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
      | NamedExpr(expr target, expr value)
      | BinOp(expr left, operator op, expr right)
      | UnaryOp(unaryop op, expr operand)
      | Lambda(arguments args, expr body)
      | IfExp(expr test, expr body, expr orelse)
      | Dict(expr* keys, expr* values)
      | Set(expr* elts)
      | ListComp(expr elt, comprehension* generators)
```

```
lambda a, b: a < b
```

```
Expr(
  value=Lambda(
    args=arguments(
      args=[
        arg(arg='a'),
        arg(arg='b')]
      ),
    body=Compare(
      left=Name(id='a'),
      ops=[Lt()],
      comparators=[
        Name(id='b')
      ]
    )
  )
)
```

Modeling Language

```
lambda a, b: a < b
```

Abstract Syntax Tree

```
Expr(  
  value=Lambda(  
    args=arguments(  
      args=[  
        arg(arg='a'),  
        arg(arg='b')] ),  
    body=Compare(  
      left=Name(id='a'),  
      ops=[Lt()],  
      comparators=[  
        Name(id='b')  
      ]  
    )  
  )  
)
```

Modeling Language

```
lambda a, b: a < b
```

Abstract Syntax Tree

```
Expr(  
  value=Lambda(  
    args=arguments(  
      args=[  
        arg(arg='a'),  
        arg(arg='b')] ),  
    body=Compare(  
      left=Name(id='a'),  
      ops=[Lt()],  
      comparators=[  
        Name(id='b')  
      ]  
    )  
  )  
)
```

Tokens (bert-base-uncased)

```
Tokens: [  
  '[CLS]',  
  'lambda',  
  'a',  
  ',',  
  ',',  
  'b',  
  ':',  
  'a',  
  '<',  
  'b',  
  '[SEP]'  
)
```

Modeling Language

```
lambda a, b: a < b
```

Abstract Syntax Tree

```
Expr(  
  value=Lambda(  
    args=arguments(  
      args=[  
        arg(arg='a'),  
        arg(arg='b')] ),  
    body=Compare(  
      left=Name(id='a'),  
      ops=[Lt()],  
      comparators=[  
        Name(id='b')  
      ]  
    )  
  )  
)
```

Tokens (bert-base-uncased)

```
Tokens: [  
  '[CLS]',  
  'lambda',  
  'a',  
  ',',  
  'b',  
  ':',  
  'a',  
  '<',  
  'b',  
  '[SEP]'  
]
```

Tokens (gpt2)

```
Tokens: [  
  'lambda',  
  'Ġa',  
  'Ġ,',  
  'Ġb',  
  'Ġ:',  
  'Ġa',  
  'Ġ<',  
  'Ġb'  
]
```

Come up with a python program that tokenizes a string (similar to what NLP tokenizer would do) please import the relevant tokenizers from, say, huggingface or so on

Sure! I'll show you a Python program that uses the 🤗 **Hugging Face** `transformers` library to tokenize a string. This library provides pre-trained tokenizers that mimic what's used in modern NLP models (BERT, GPT-2, etc.).

Here's an example:

```
python 📋 Copy code  
  
# Install the library if not already installed:  
# pip install transformers  
  
from transformers import AutoTokenizer  
  
def tokenize_text(text, model_name="bert-base-uncased"):  
    """  
    Tokenizes the input text using the Hugging Face tokenizer for the given model.  
  
    Args:  
        text (str): Input string to tokenize  
        model_name (str): Name of the pretrained model tokenizer to use  
  
    Returns:  
        dict: A dictionary containing tokenized ids, attention masks, etc.  
    """
```

Abstract Syntax Tree

```
Expr(  
  value=Lambda(  
    args=arguments(  
      args=[  
        arg(arg='a'),  
        arg(arg='b')] ),  
    body=Compare(  
      left=Name(id='a'),  
      ops=[Lt()],  
      comparators=[  
        Name(id='b')  
      ]  
    )  
  )  
)
```

lambda a, b: a < b

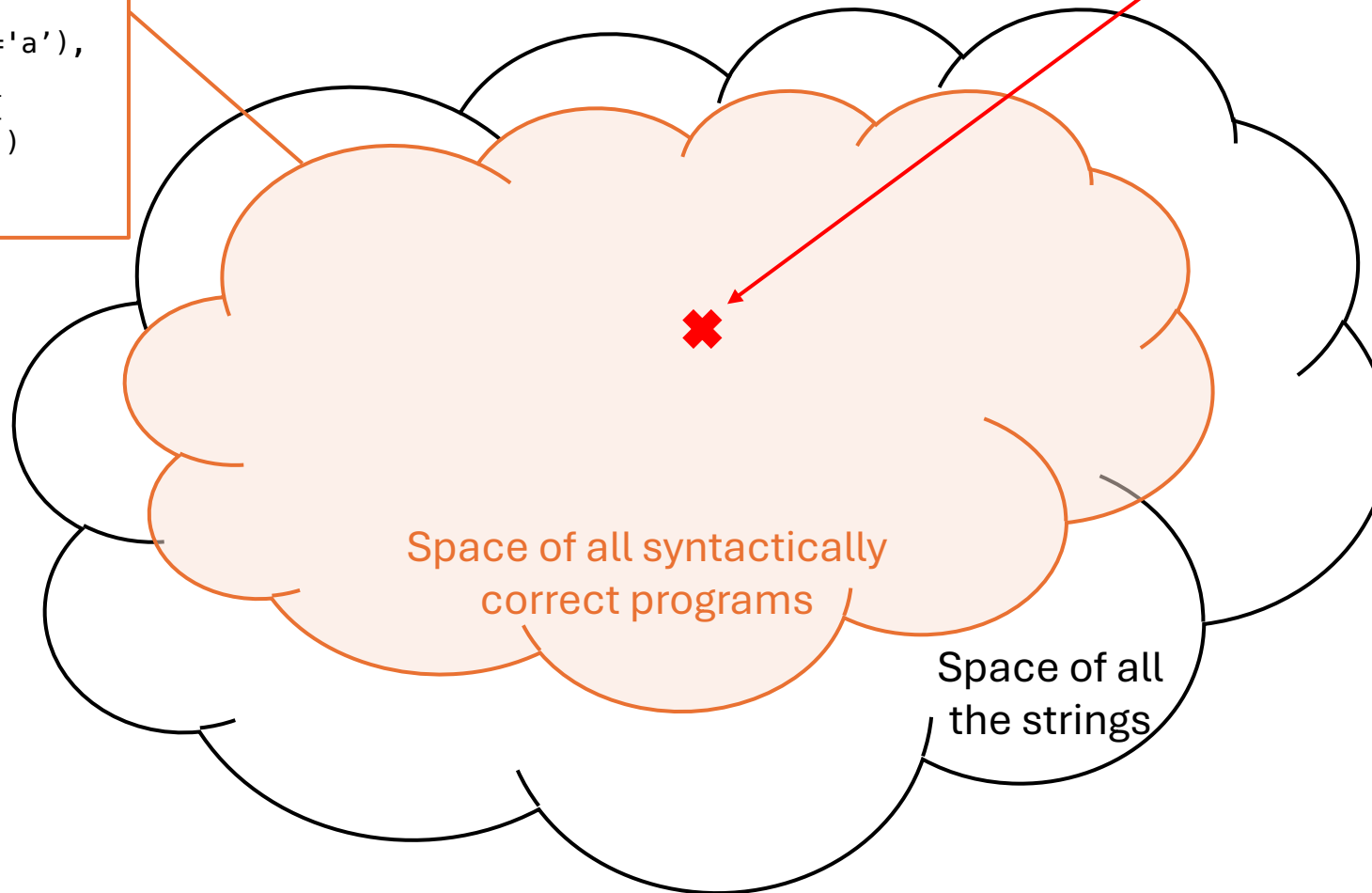
Desired Program

Tokens (bert-base-uncased)

```
Tokens: [  
  '[CLS]',  
  'lambda',  
  'a',  
  ',',  
  'b',  
  ',',  
  'a',  
  '<',  
  'b',  
  '[SEP]'  
]
```

Tokens (gpt2)

```
Tokens: [  
  'lambda',  
  'Ga',  
  ',',  
  'Gb',  
  ',',  
  'G',  
  'Ga',  
  'G<',  
  'Gb'  
]
```



Abstract Syntax Tree

```
Expr(  
  value=Lambda(  
    args=arguments(  
      args=[  
        arg(arg='a'),  
        arg(arg='b')] ),  
    body=Compare(  
      left=Name(id='a'),  
      ops=[Lt()],  
      comparators=[  
        Name(id='b')  
      ]  
    )  
  )  
)
```

lambda a, b: a < b

Desired Program

Tokens (bert-base-uncased)

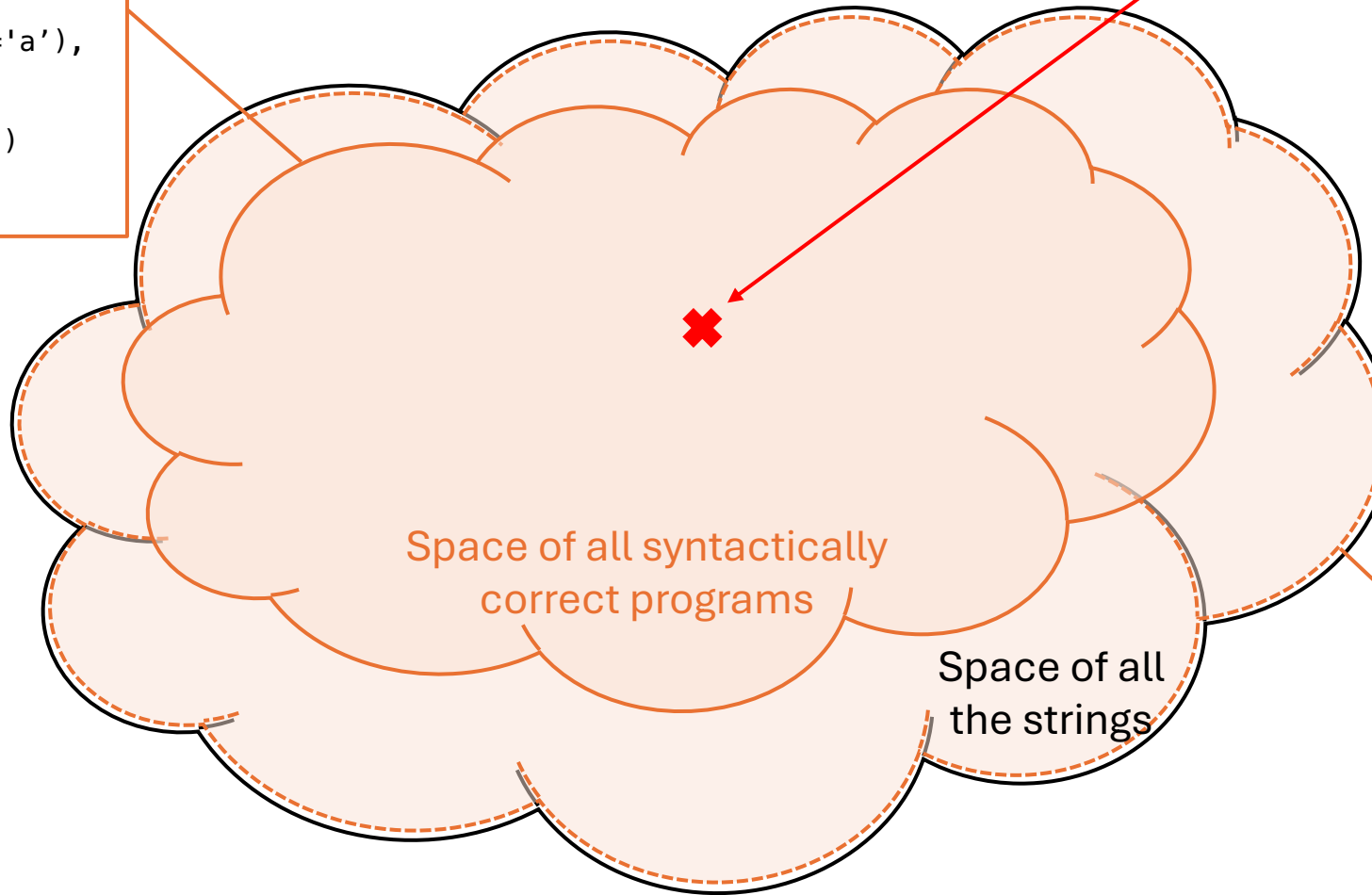
```
Tokens: [  
  '[CLS]',  
  'lambda',  
  'a',  
  ',',  
  'b',  
  ',',  
  'a',  
  '<',  
  'b',  
  '[SEP]'  
]
```

Tokens (gpt2)

```
Tokens: [  
  'lambda',  
  'Ga',  
  ',',  
  'Gb',  
  ',',  
  'G',  
  'Ga',  
  'G<',  
  'Gb'  
]
```

Space of all syntactically
correct programs

Space of all
the strings



Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Modeling Language

Problem Definition: **Next Token Prediction**

Input: Prefix $t_1, t_2, \dots, t_{i-1}$

Output: Next Token t_i

Problem Example: **Auto-completion**

Input: Prefix `for i in ???`

Output: Next Token `range`

Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Problem Example: **Auto-completion**

Input: **Prefix** for i in ???

Output: **Next Token** range

After tokenizing by GPT-2 tokenizer

$['for', 'Ġi', 'Ġin', 'Ġ']$

t_1

t_2

t_3

t_4

t_5

Modeling Language

Problem Definition: **Next Token Prediction**

Input: Prefix $t_1, t_2, \dots, t_{i-1}$

Output: Next Token t_i

Problem Example: **Auto-completion**

Input: Prefix $['for', 'Gi', 'Gin', 'G']$
 $t_1 \quad t_2 \quad t_3 \quad t_4$

Output: Next Token $range$
 t_5

Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$

Problem Example: **Auto-completion**

Input: **Prefix** $['for', 'Gi', 'Gin', 'G']$
 $t_1 \quad t_2 \quad t_3 \quad t_4$

Output: **Next Token** $range$
 t_5

Goal: Compute the probability $\Pr(range \mid for, Gi, Gin, G)$

Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



Modeling Language

N-Gram Models

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$

Solution: **N-Gram**

Idea: Approximating $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$ by only considering the $n - 1$ tokens preceding it

$$\Pr(t_i \mid t_1, t_2, \dots, t_{i-1}) \approx \Pr(t_i \mid t_{i-n+1}, \dots, t_{i-1})$$

Modeling Language

N-Gram Models

Solution: **N-Gram**

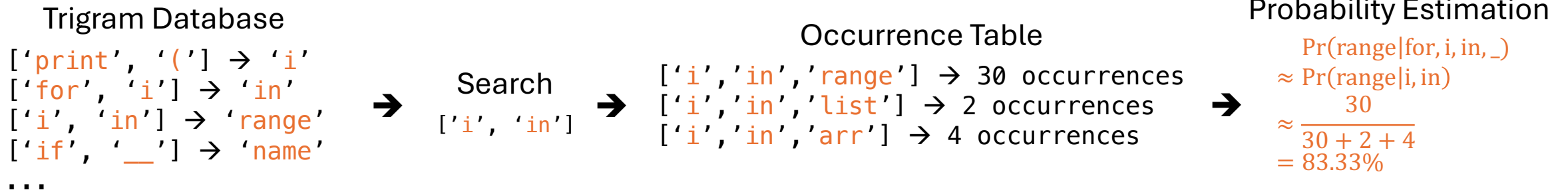
Idea: Approximating $\Pr(t_i | t_1, t_2, \dots, t_{i-1})$ by only considering the $n - 1$ tokens preceding it

$$\Pr(t_i | t_1, t_2, \dots, t_{i-1}) \approx \Pr(t_i | t_{i-n+1}, \dots, t_{i-1})$$

Problem Example: **Auto-completion**

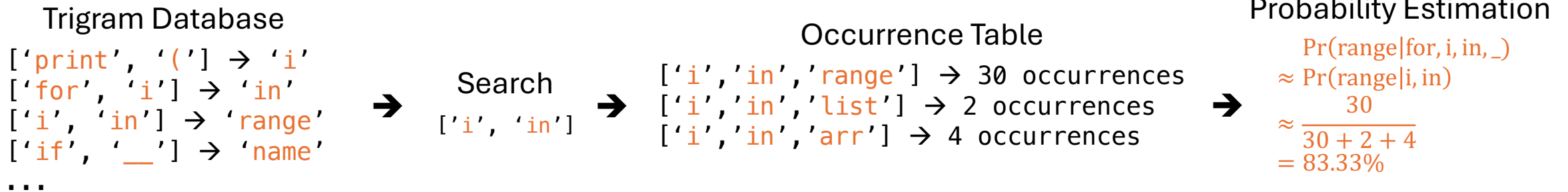
Input: **Prefix** `for i in ???`

Output: **Next Token** `range`



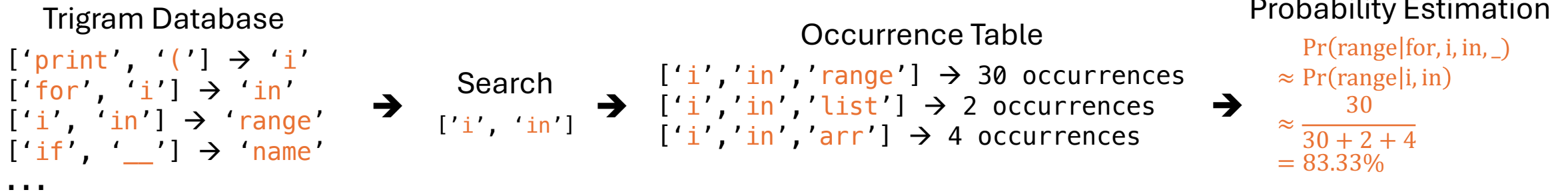
Modeling Language

N-Gram Models



Modeling Language

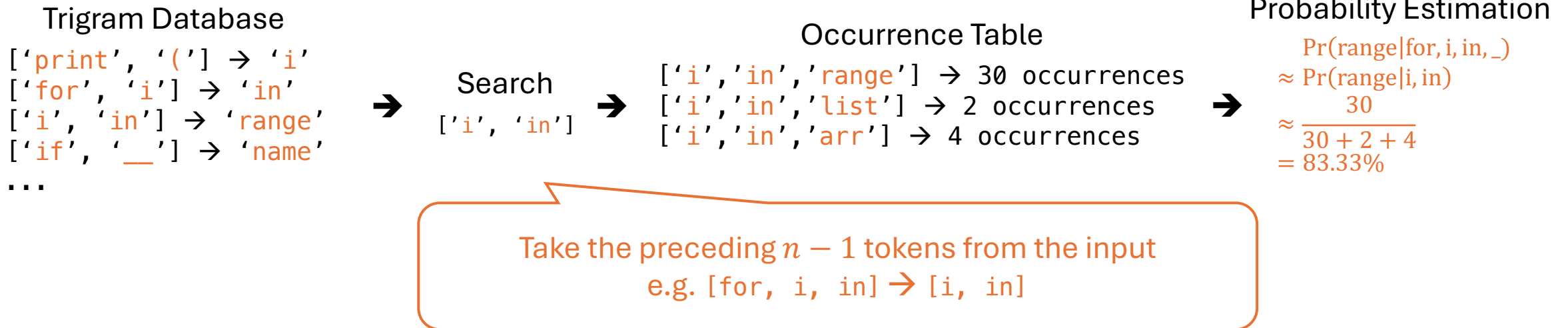
N-Gram Models



Trigram database = set of 3-tuples of consecutive tokens.
can be built from a dataset of programs

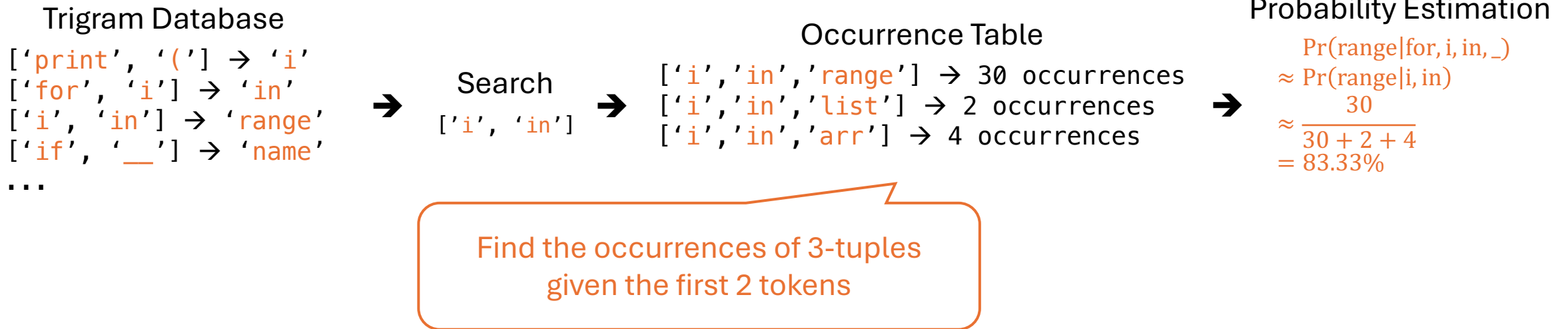
Modeling Language

N-Gram Models



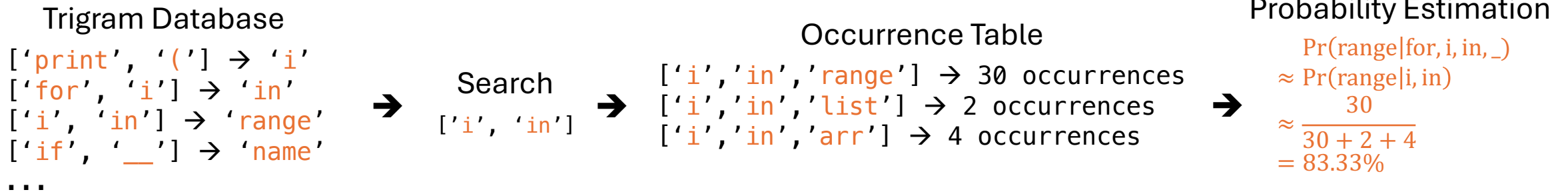
Modeling Language

N-Gram Models



Modeling Language

N-Gram Models



Apply Maximum Likelihood Estimation (MLE)

$$\Pr(\text{range}|i, \text{in}) = \frac{\text{Count}(i, \text{in}, \text{range})}{\sum_w \text{Count}(i, \text{in}, w)}$$

On the Naturalness of Software

Abram Hindle, Earl Barr, Mark Gabel, Zhendong Su, Prem Devanbu
devanbu@cs.ucdavis.edu

of NLP is this: *natural language may be complex and admit a great wealth of expression, but what people write and say is largely regular and predictable.*

Our central hypothesis is that the same argument applies to software:

Programming languages, in theory, are complex, flexible and powerful, but the programs that real people actually write are mostly simple and rather repetitive, and thus they have usefully predictable statistical properties that can be captured in statistical language models and leveraged for software engineering tasks.

These models are estimated from a corpus using simple maximum-likelihood based frequency-counting of token sequences. Thus, if “*” is a wildcard, we ask, how relatively often are the tokens a_1, a_2, a_3 followed by a_4 :

$$p(a_4|a_1a_2a_3) = \frac{\text{count}(a_1a_2a_3a_4)}{\text{count}(a_1a_2a_3*)}$$

In practice, estimation of n -gram models is quite a bit more complicated. The main difficulties arise from data sparsity, *i.e.*, the richness of the model in comparison to the available data. For example, with 10^4 token vocabulary, a trigram model must estimate 10^{12} coefficients. Some trigrams may never occur in one corpus, but may in fact occur elsewhere. This will

On the Naturalness of Software

Abram Hindle, Earl Barr, Mark Gabel, Zhendong Su, Prem Devanbu
devanbu@cs.ucdavis.edu

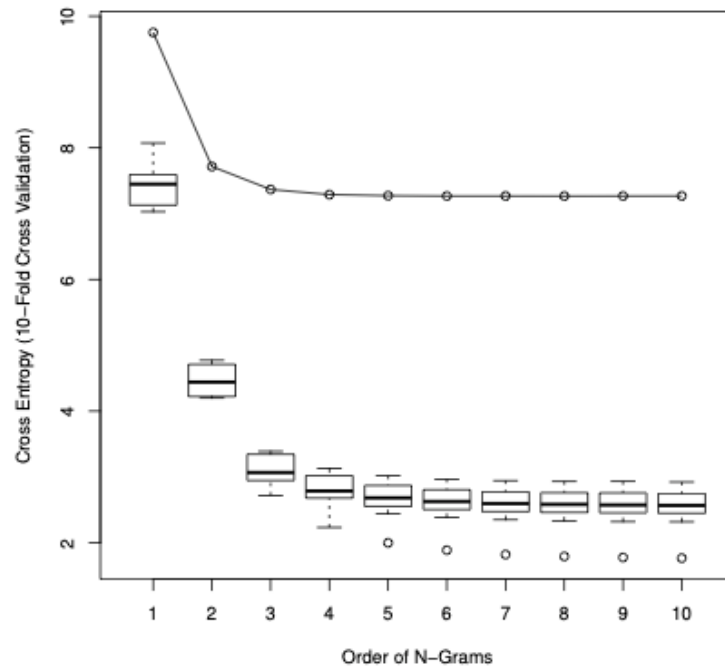


Figure 1: Comparison of English Cross-Entropy versus the Code Cross Entropy of 10 projects.

results are in Figure 1. The single line above is the average over the 10 folds for the English corpus, beginning at about 10 bits for unigram models, and trailing down to under 8 bits for 10-gram models. The average self cross-entropy for the 10 projects are shown below in boxplots, one for each order from unigram models to 10-gram models. Several observations can be made. First, software unigram entropy is much lower than might be expected from a uniform distribution over unique tokens, because token frequencies are obviously very skewed.

Second, *cross-entropy declines rapidly with n-gram order*, saturating around tri- or 4-grams. The similarity in the decline in English and the Java projects is striking. This decline suggests that there is as much of *repetitive local context that is being captured by the language model* in Java programs, as it is in the English corpora. We take this to be highly encouraging: the ability to model the regularity of the local context in natural languages has proven to be extremely valuable in statistical natural language processing; we hope (and in fact,

On the Naturalness of Software

Abram Hindle, Earl Barr, Mark Gabel, Zhendong Su, Prem Devanbu
devanbu@cs.ucdavis.edu

IV. SUGGESTING THE NEXT TOKEN

The strikingly low entropy (between 3 and 4 bits) produced by the smoothed n-gram model indicates that even at the local token-sequence level, there is a high degree of “naturalness”. If we make 8-16 guesses ($2^3 - 2^4$) as to what the next token is, we may very well guess the right one!

Eclipse Suggestion Plug-in We built an Eclipse plug-in to test this idea. Eclipse, like many other IDEs, has a built-in *suggestion* engine that suggests a next token whenever it can. Eclipse (and other IDEs) suggestions are typically based on type information available in context. We conjectured that corpus-based n-gram models suggestion engine (for brevity, $\mathcal{NGSE}$) could enhance eclipse’s built-in suggestion engine (for brevity, $\mathcal{ECSE}$) by offering tokens that tend to *naturally* follow from preceding ones in the relevant corpus.

Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



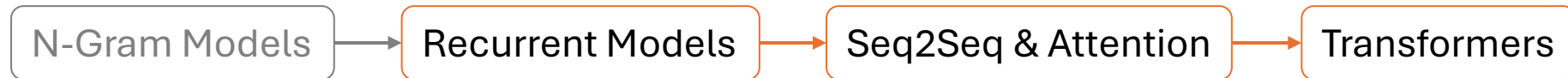
Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



Modeling Language

Recurrent Models

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$

Modeling Language

Recurrent Models

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$

Problems with N-Grams:

- Can't remember beyond a fixed window
- Data sparsity (unseen n-grams)
- Ignores semantic structures



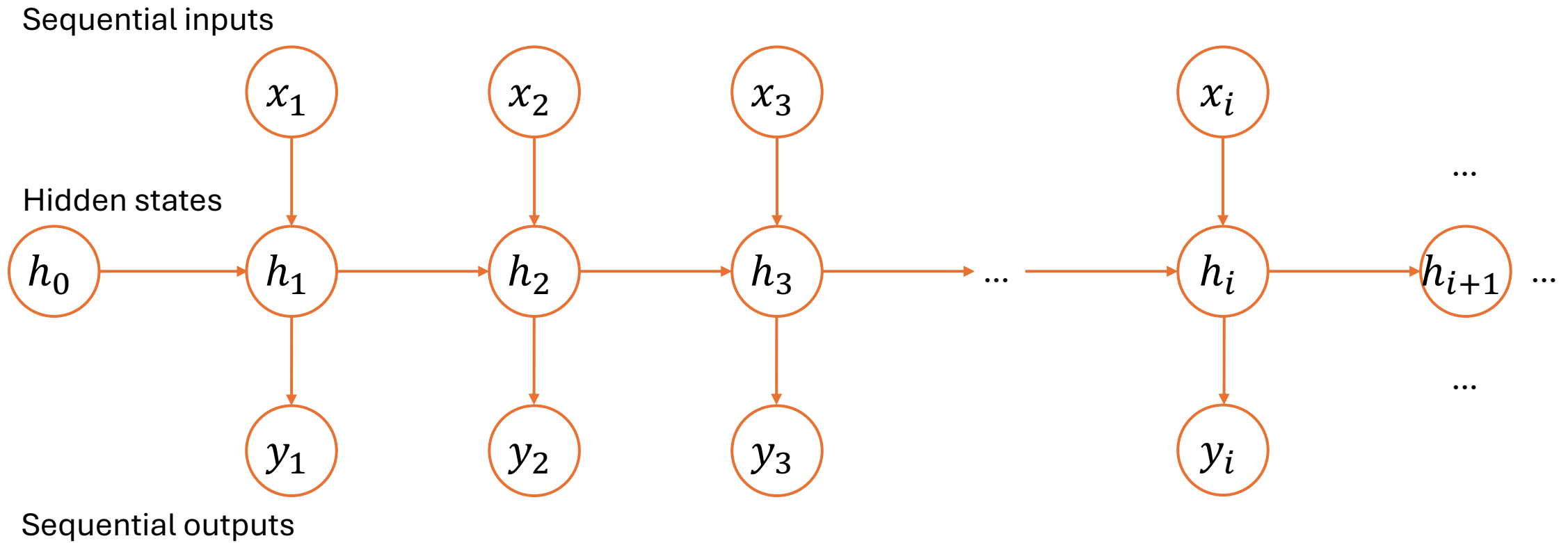
Recurrent Models

Motivation:

- A model that can remember context of arbitrary length
- The context can be updated dynamically

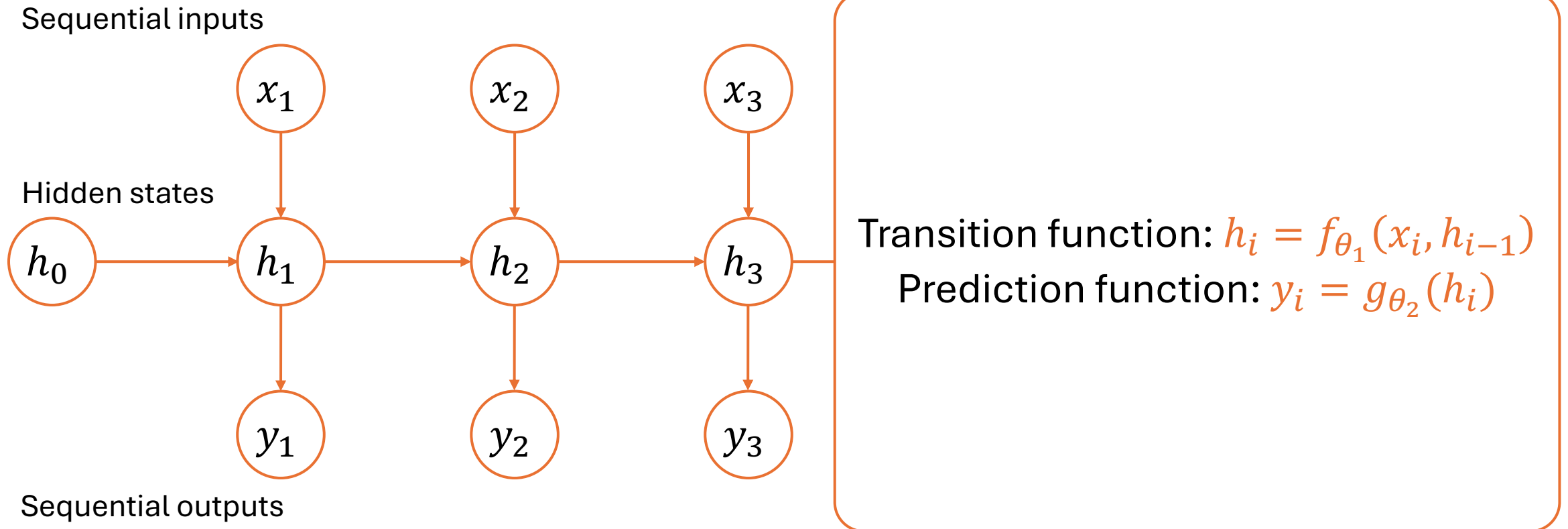
Modeling Language

Recurrent Models



Modeling Language

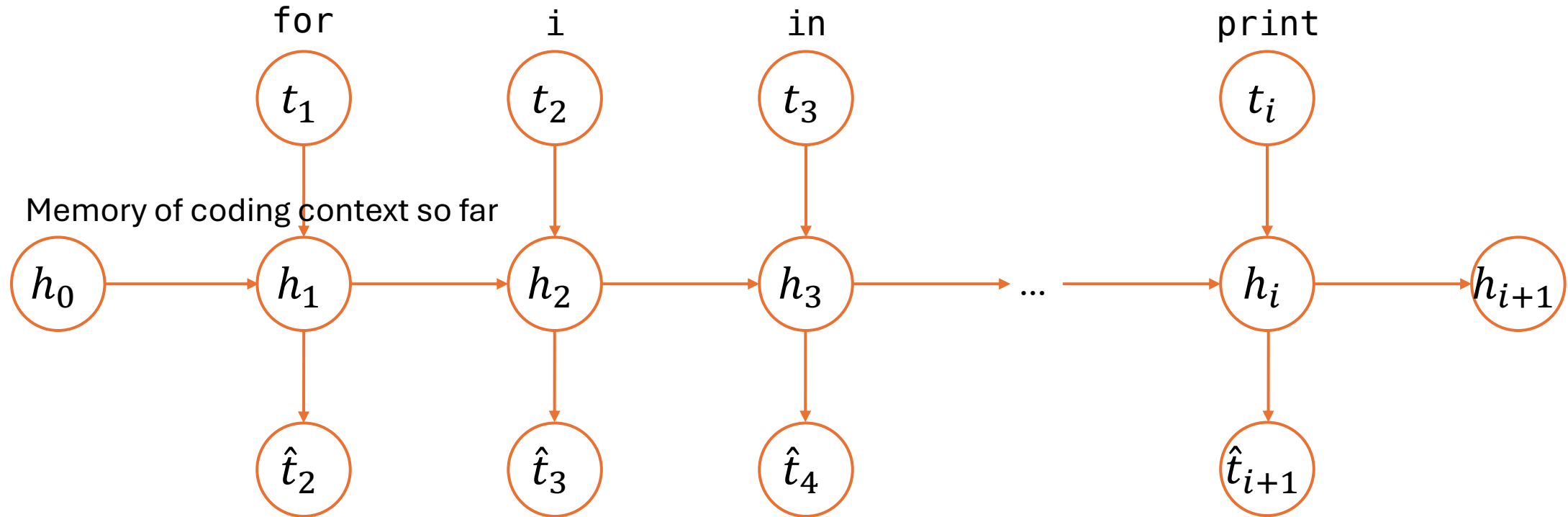
Recurrent Models



Modeling Language

Recurrent Models

Input tokens

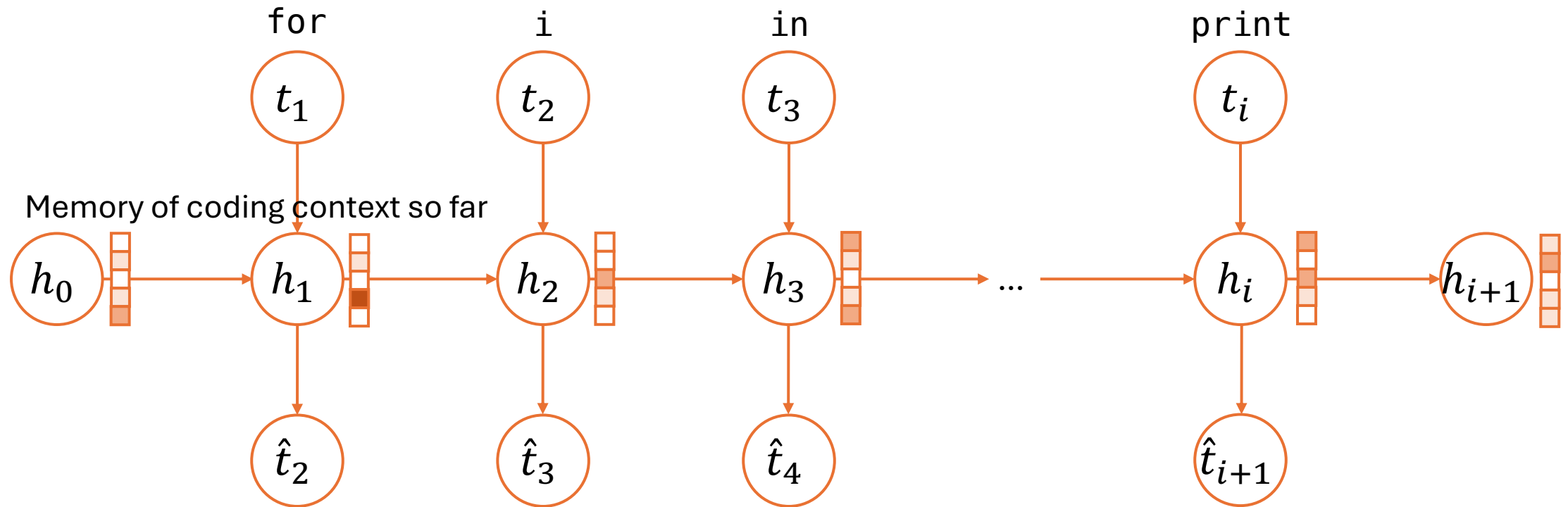


Predicted next tokens

Modeling Language

Recurrent Models

Input tokens

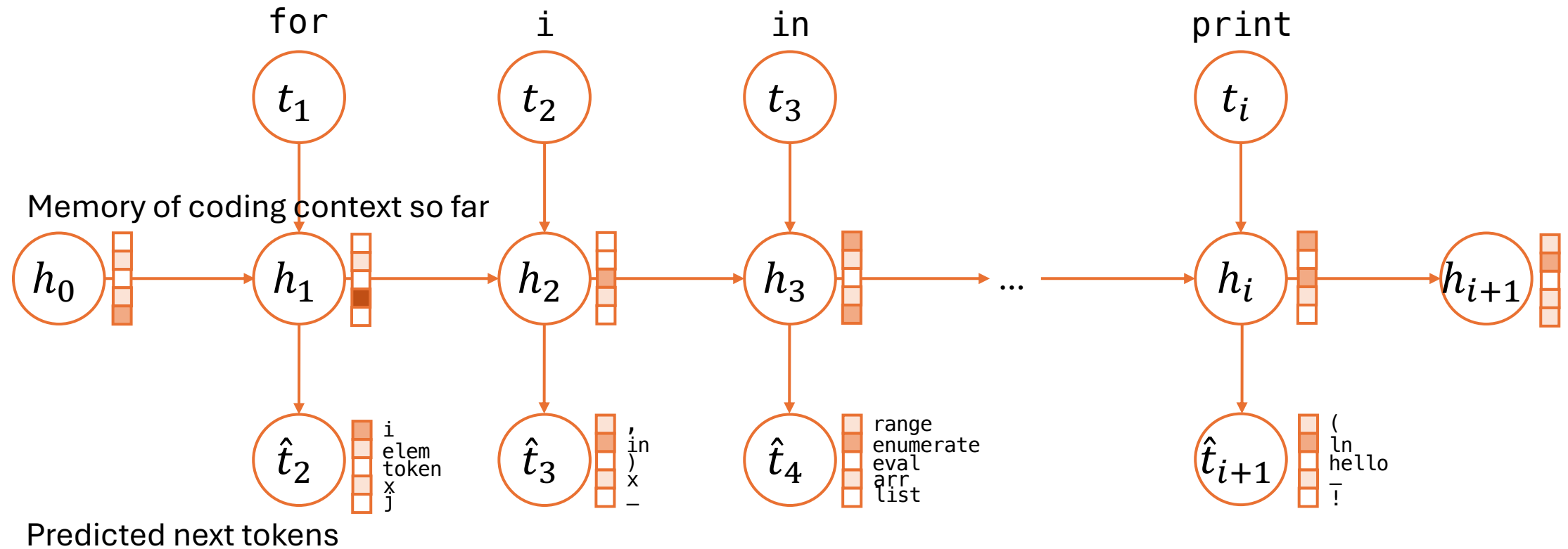


Predicted next tokens

Modeling Language

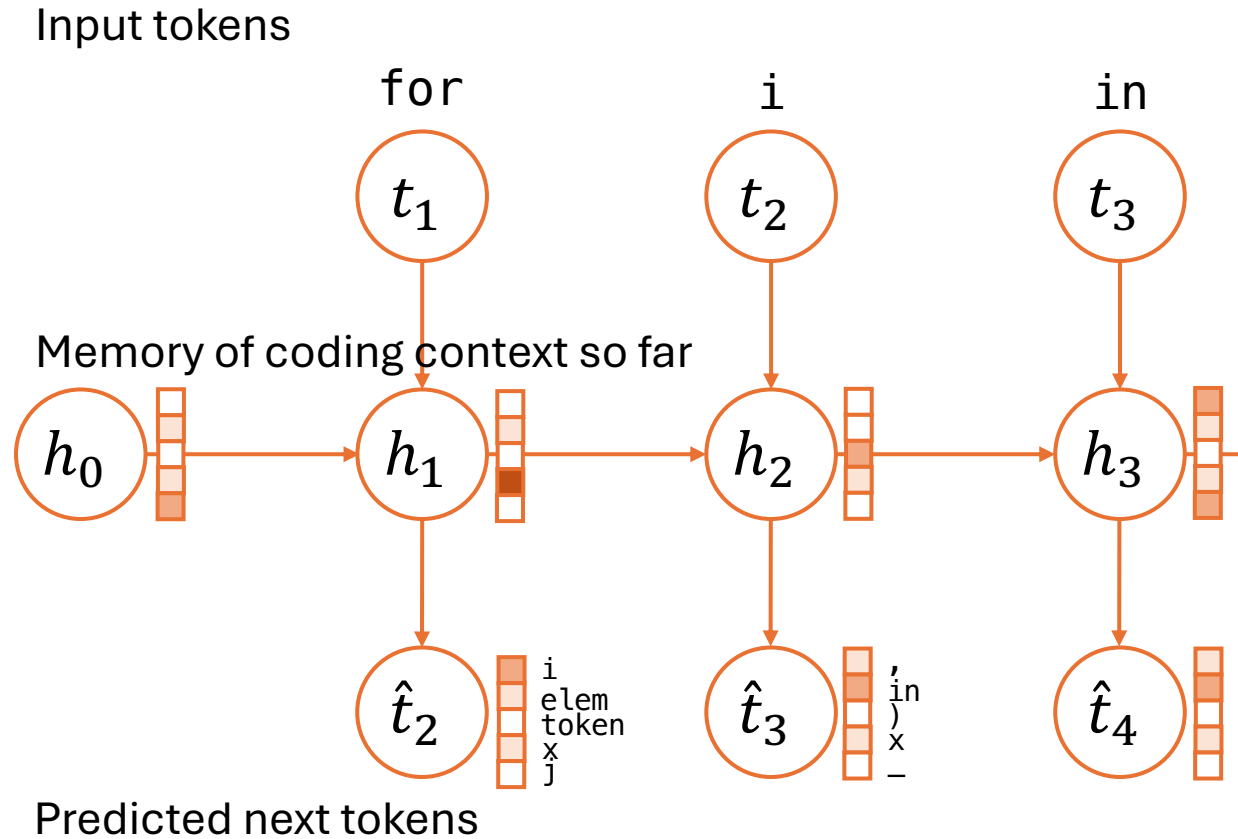
Recurrent Models

Input tokens



Modeling Language

Recurrent Models



Memory/context encoder:

$$h_i = f_{\theta_1}(t_i, h_{i-1})$$

Next token distribution:

$$\hat{t}_{i+1} = \Pr(t_{i+1} | t_1, \dots, t_i) = g_{\theta_2}(h_i)$$

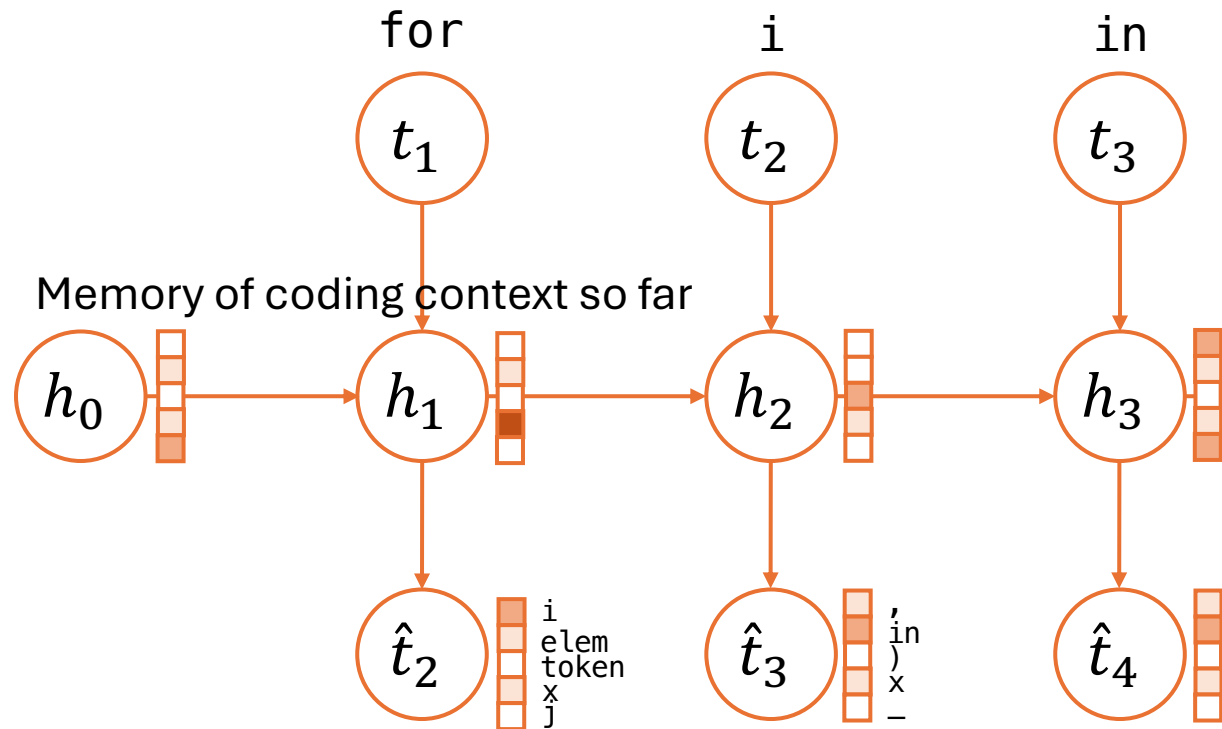
Draw the predicted next token:

$$t_{i+1} \sim \Pr(\cdot | t_1, \dots, t_i)$$

Modeling Language

Recurrent Models

Input tokens



Predicted next tokens

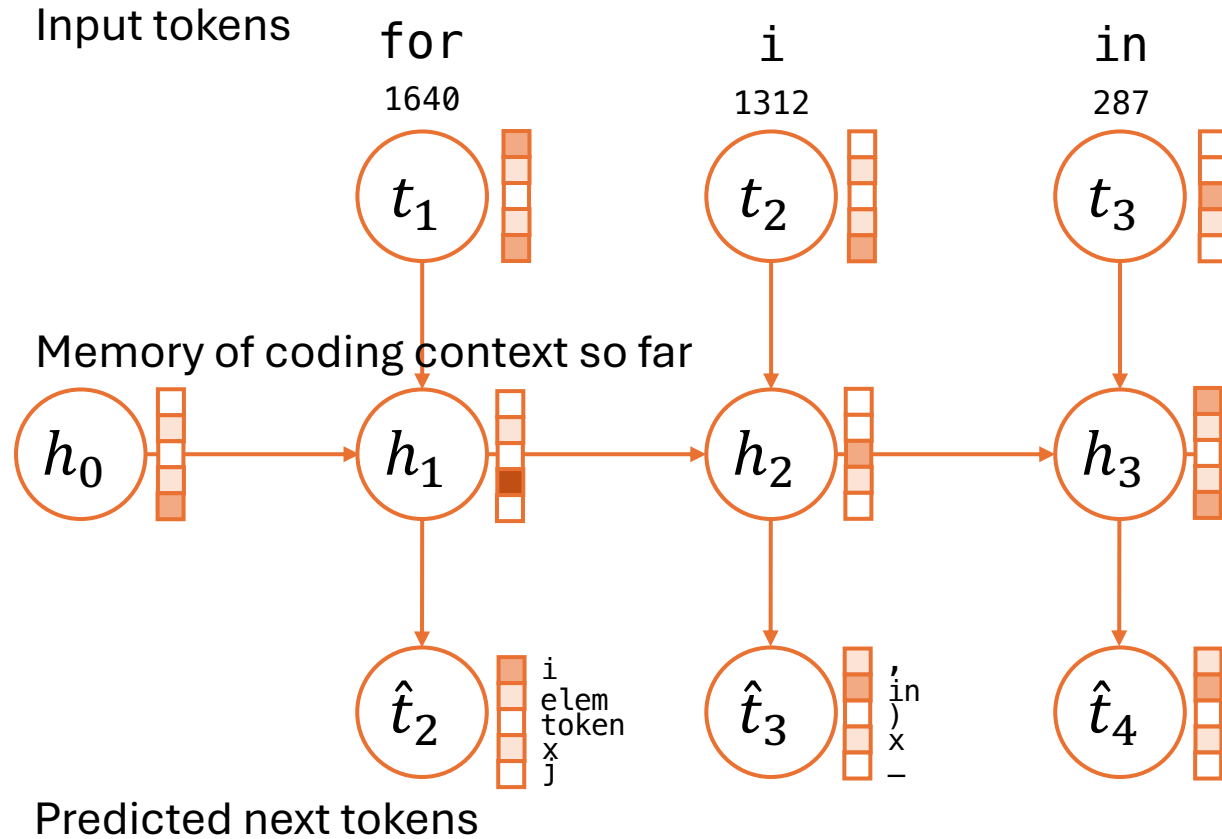
```
text = "for i in range(10):\nprint(i)"
result = tokenize_text(text)
print("Tokens:", result["tokens"])
print("Token IDs:", result["encoded"]["input_ids"])
```

Tokens: ['for', 'i', 'in', 'range',
'(', '10', '):', ' ', 'print', '(',
'i', ')']

Token IDs: tensor([[1640, 1312, 287,
2837, 7, 940, 2599, 198, 4798,
7, 72, 8]])

Modeling Language

Recurrent Models



```
text = "for i in range(10):\nprint(i)"
result = tokenize_text(text)
print("Tokens:", result["tokens"])
print("Token IDs:", result["encoded"]["input_ids"])
```

Tokens: ['for', 'i', 'in', 'range',
'(', '10', '):', ' ', 'print', '(',
'i', ')']

Token IDs: tensor([[1640, 1312, 287,
2837, 7, 940, 2599, 198, 4798,
7, 72, 8]])

Toward Deep Learning Software Repositories

Martin White, Christopher Vendome, Mario Linares-Vásquez, and Denys Poshyvanyk

Department of Computer Science

College of William and Mary

Williamsburg, Virginia 23187-8795

Email: {mgwhite, cvendome, mlinarev, denys}@cs.wm.edu

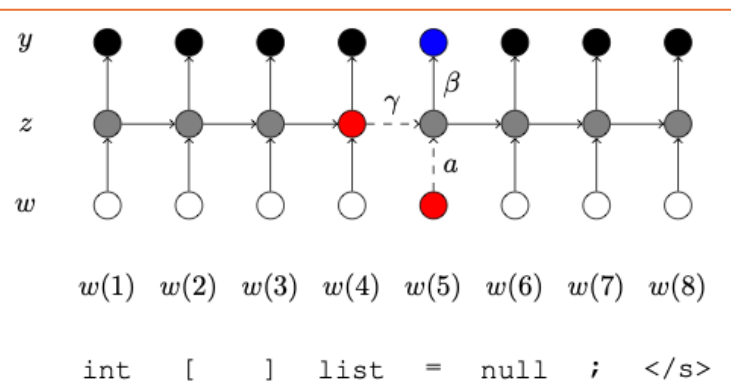


Fig. 1. RNN UNFOLDED IN TIME. The depth of a RNN is evident when the recurrence is unfolded in time. Time steps correspond to software tokens $w(t)$ in a corpus, where $w(0) = \langle s \rangle$. Each node in the figure represents a vector of units. White nodes are one-hot token representations; gray nodes are continuous-valued, hidden states; black nodes represent posterior distributions over the vocabulary. Units in the state vectors (gray nodes) compute their activation as a function of the current token and the previous state. Regarding the depth in this notional model, $y(8)$ is a function of $w(8), z(7)$, yet $z(7)$ is a function of $w(7), z(6)$, etc. Hence, predictions are informed by processing data in the past using *multiple levels of nonlinear transformations*.

TABLE IV. TOP-K ACCURACY (%)

Model	Top-1	Top-5	Top-10
Interpolated 8-gram	49.7	71.3	78.1
Interpolated 8-gram 100-cache	4.8	69.5	78.5
RNN _s -(400, 5)-1	61.1	78.4	81.4
RNN _d -(300, 20)-5	72.2	88.4	92.0

Modeling Language

Recurrent Models

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$

Solution: **Recurrent Neural Network** (RNN, LSTM, BiLSTM)

Idea: Compute $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$ by employing a recurrent neural network

Memory/context encoder:

$$h_i = f_{\theta_1}(t_i, h_{i-1})$$

Next token distribution:

$$\hat{t}_{i+1} = \Pr(t_{i+1} \mid t_1, \dots, t_i) = g_{\theta_2}(h_i)$$

Draw the predicted next token:

$$t_{i+1} \sim \Pr(\cdot \mid t_1, \dots, t_i)$$

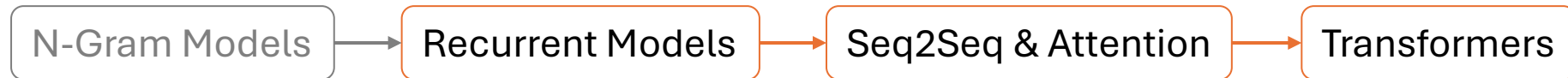
Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



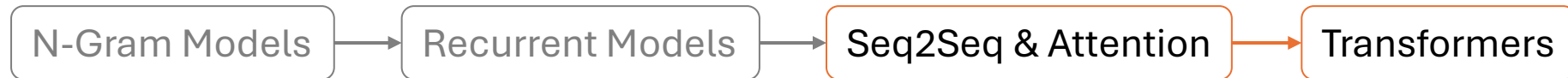
Modeling Language

Problem Definition: **Next Token Prediction**

Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



Modeling Language

Seq2Seq & Attention

Problem Definition: “**Sequence-to-Sequence Translation**”

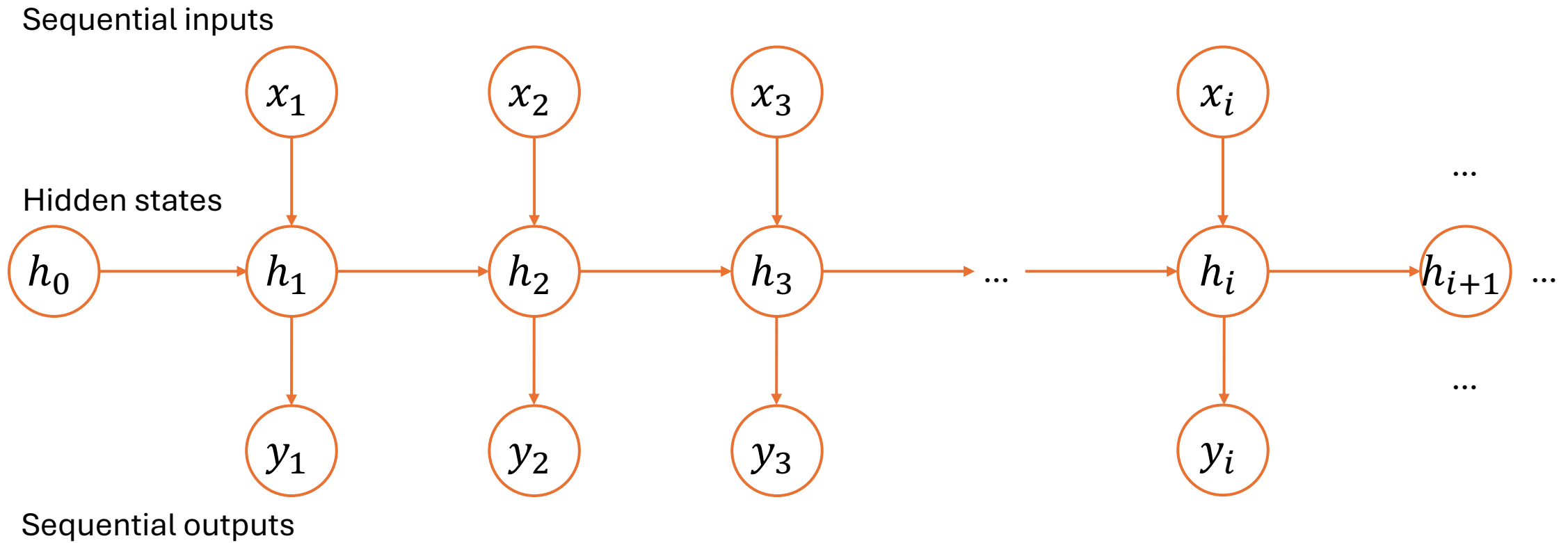
Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$

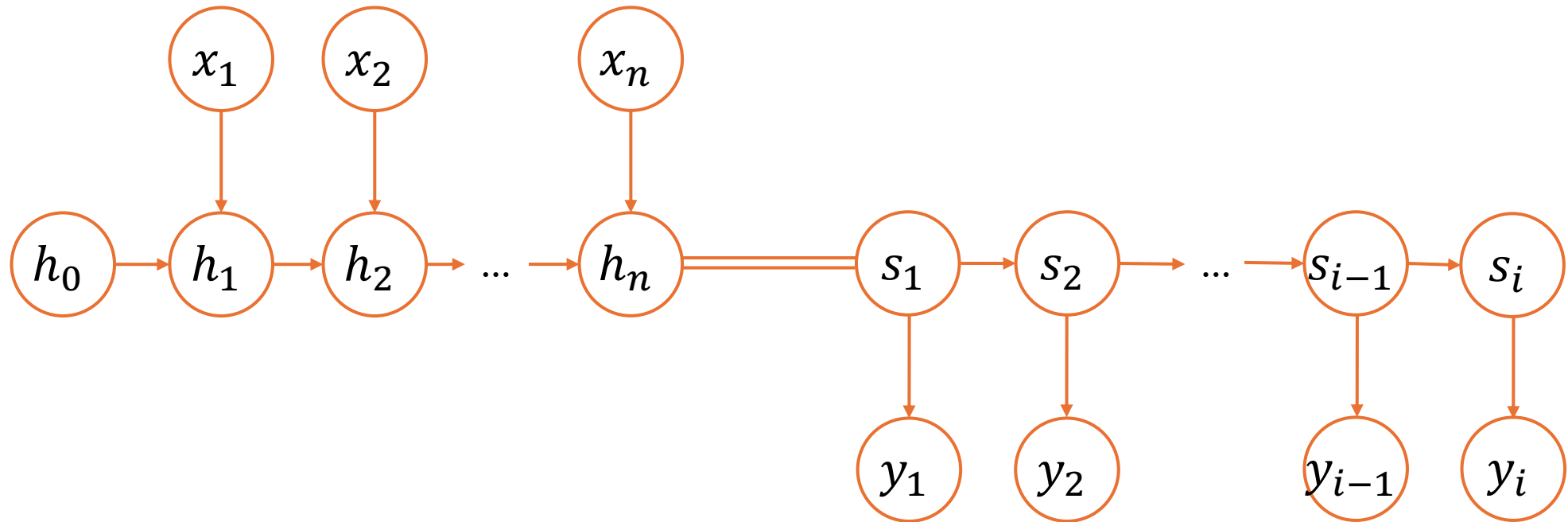
Modeling Language

Recurrent Models



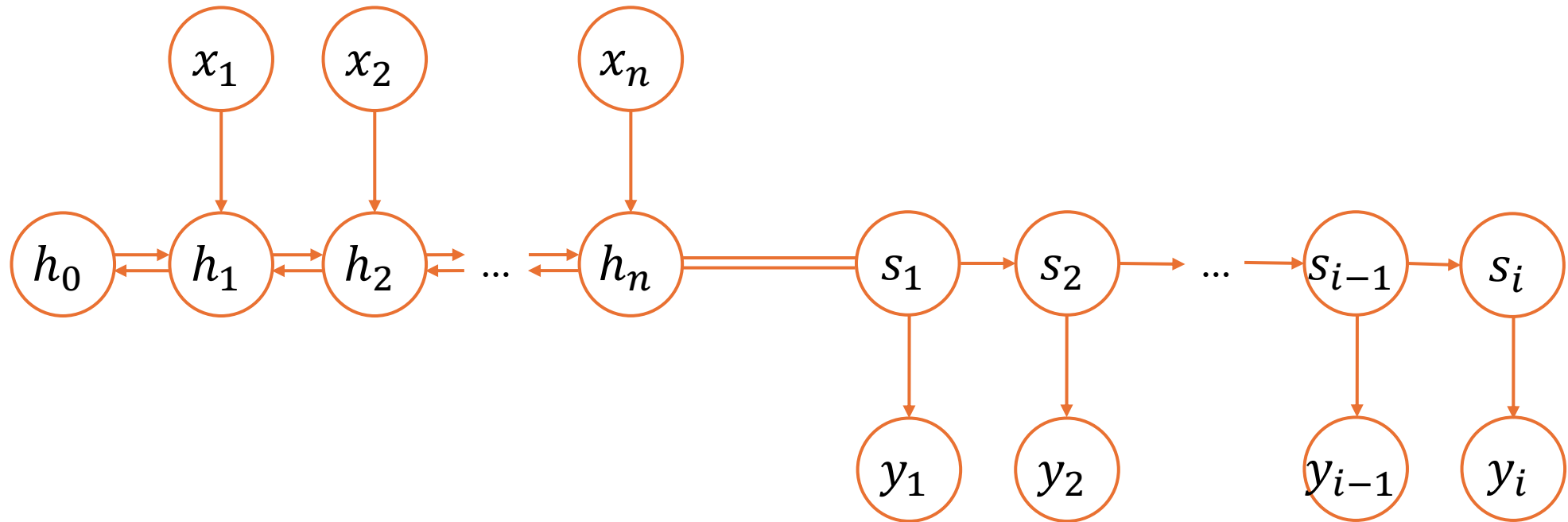
Modeling Language

Seq2Seq Models



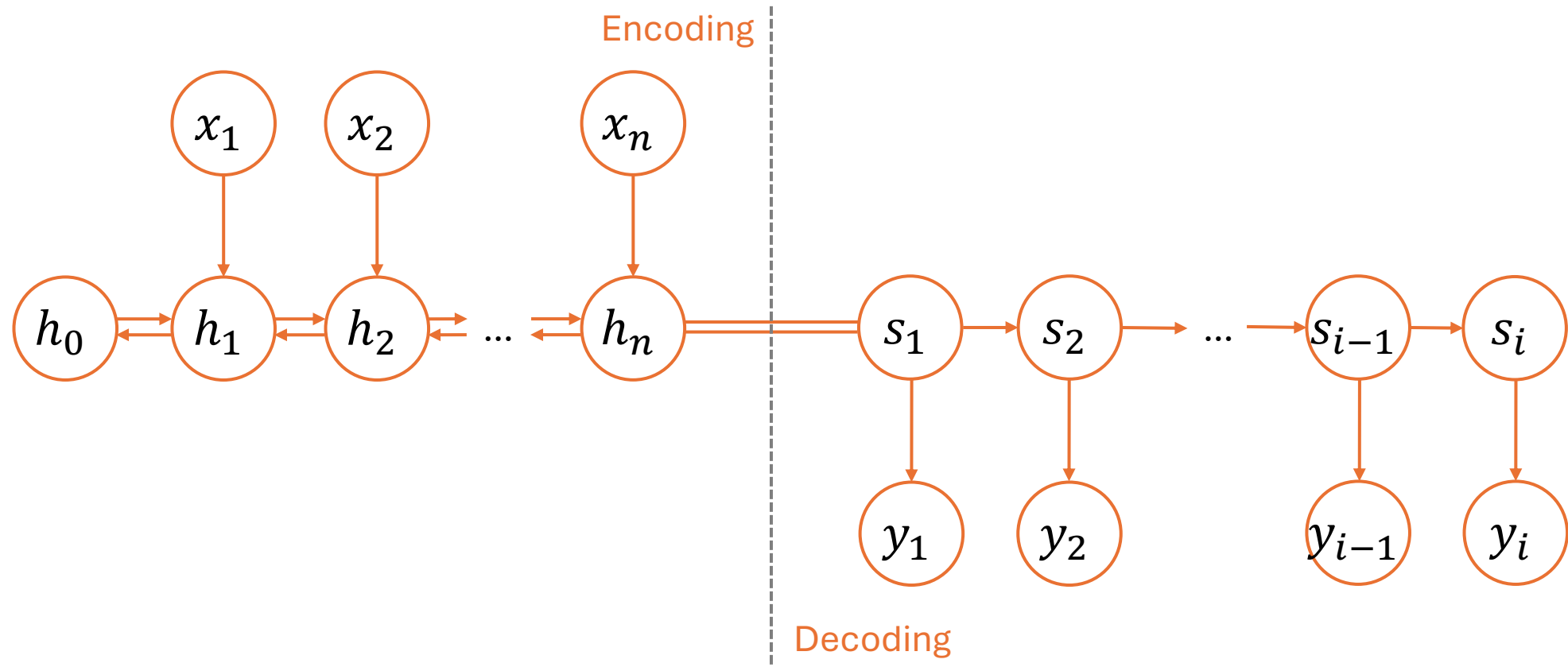
Modeling Language

Seq2Seq Models



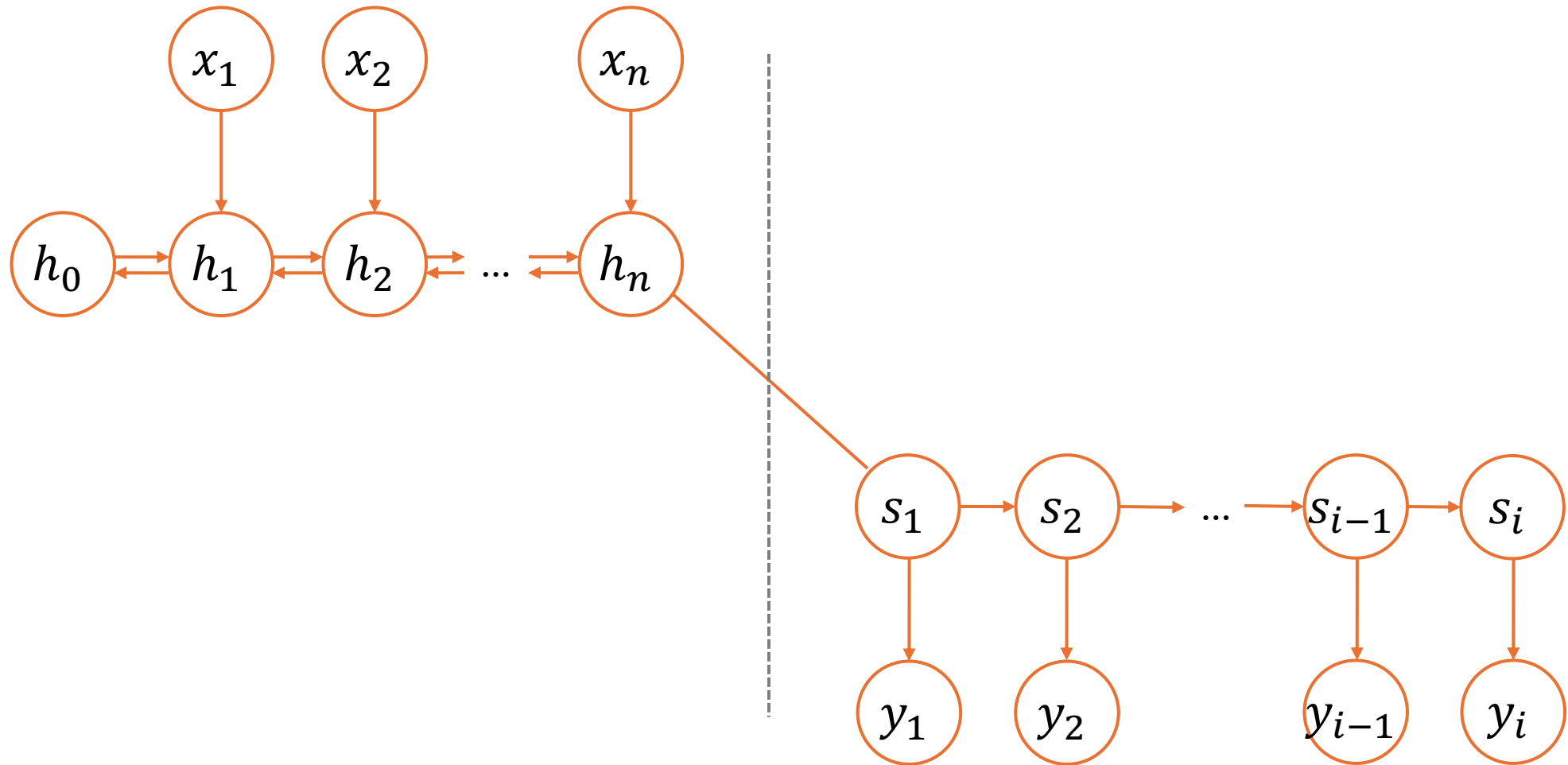
Modeling Language

Seq2Seq Models



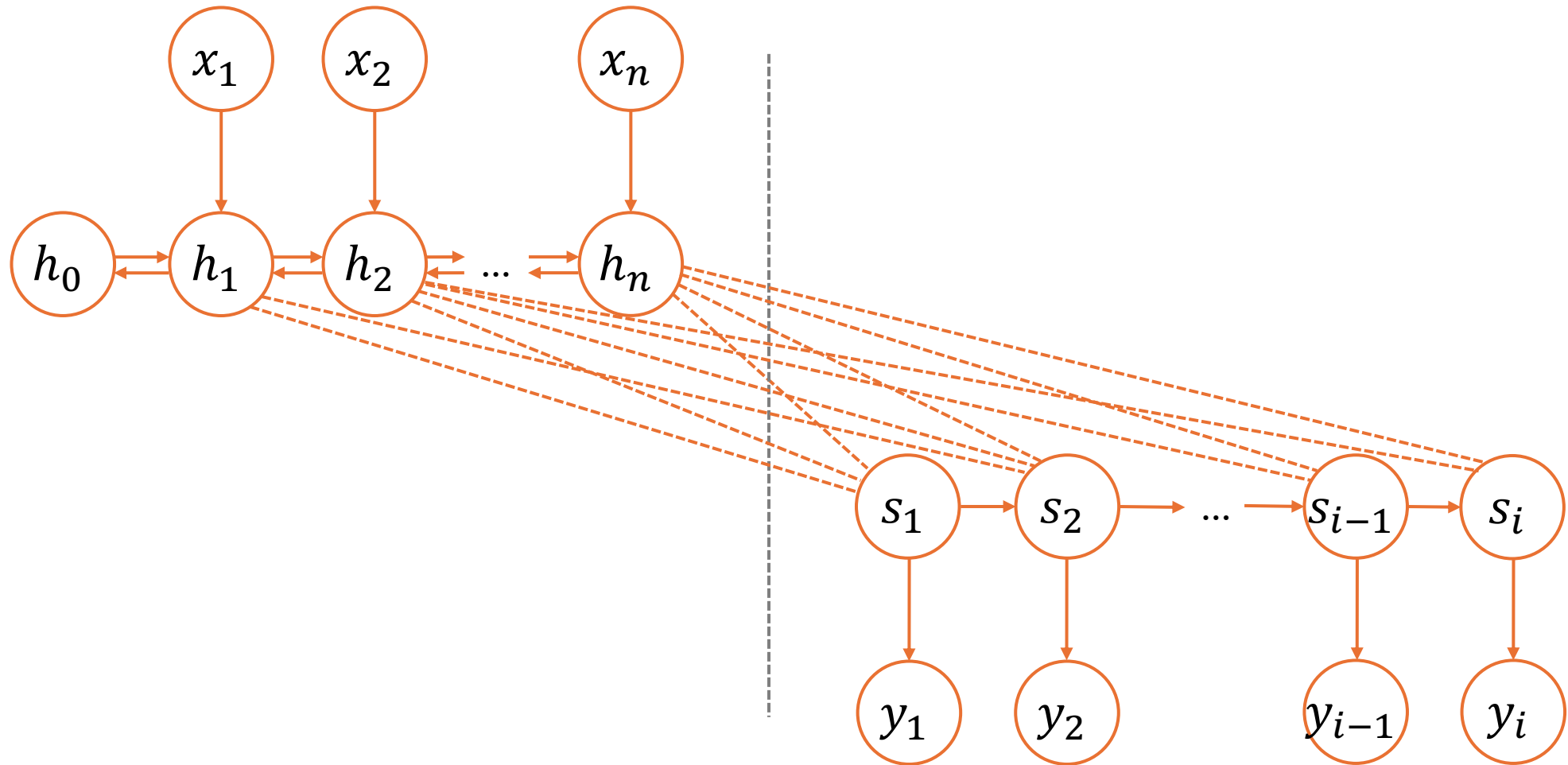
Modeling Language

Seq2Seq Models...



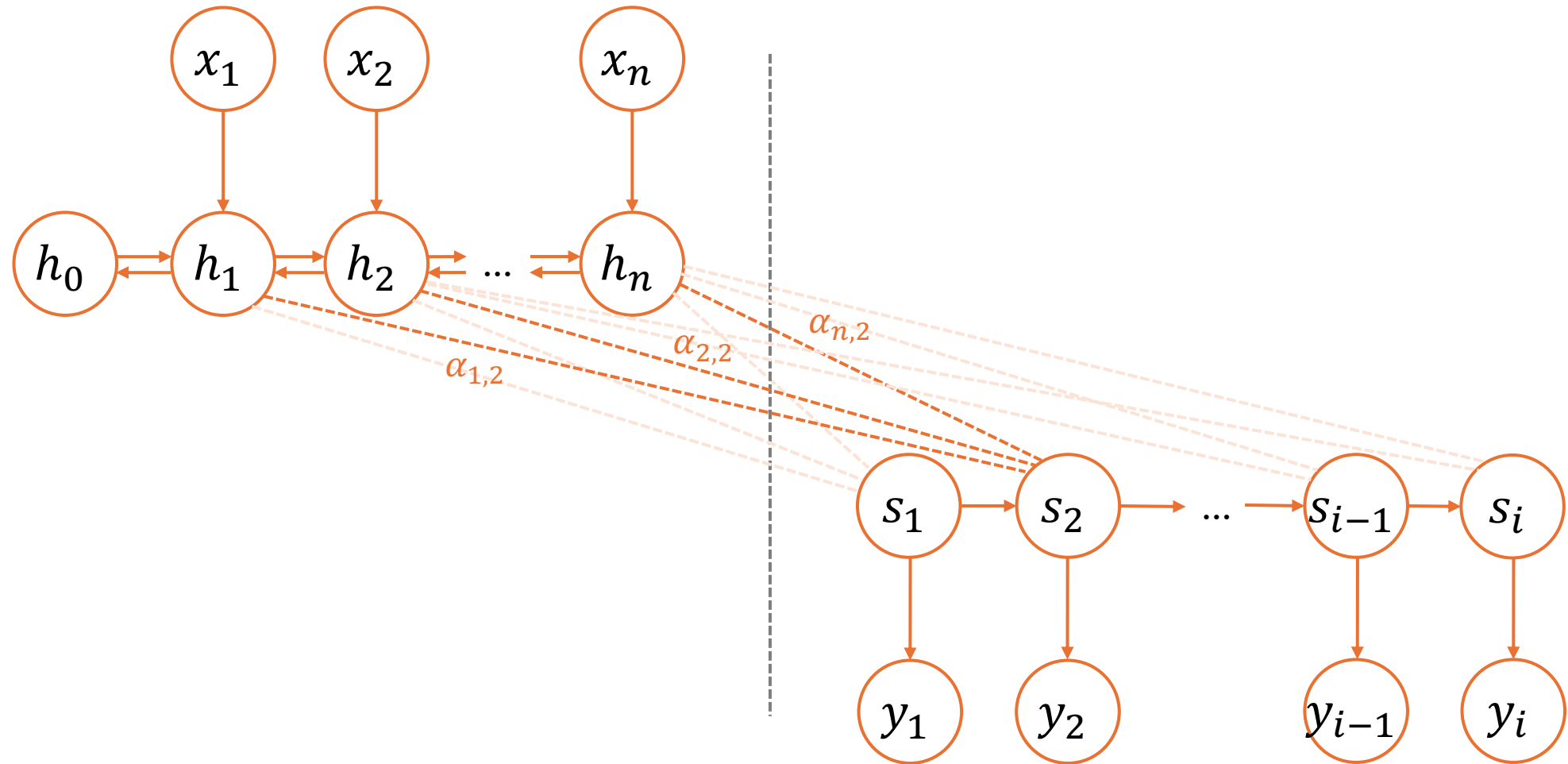
Modeling Language

Seq2Seq & Attention



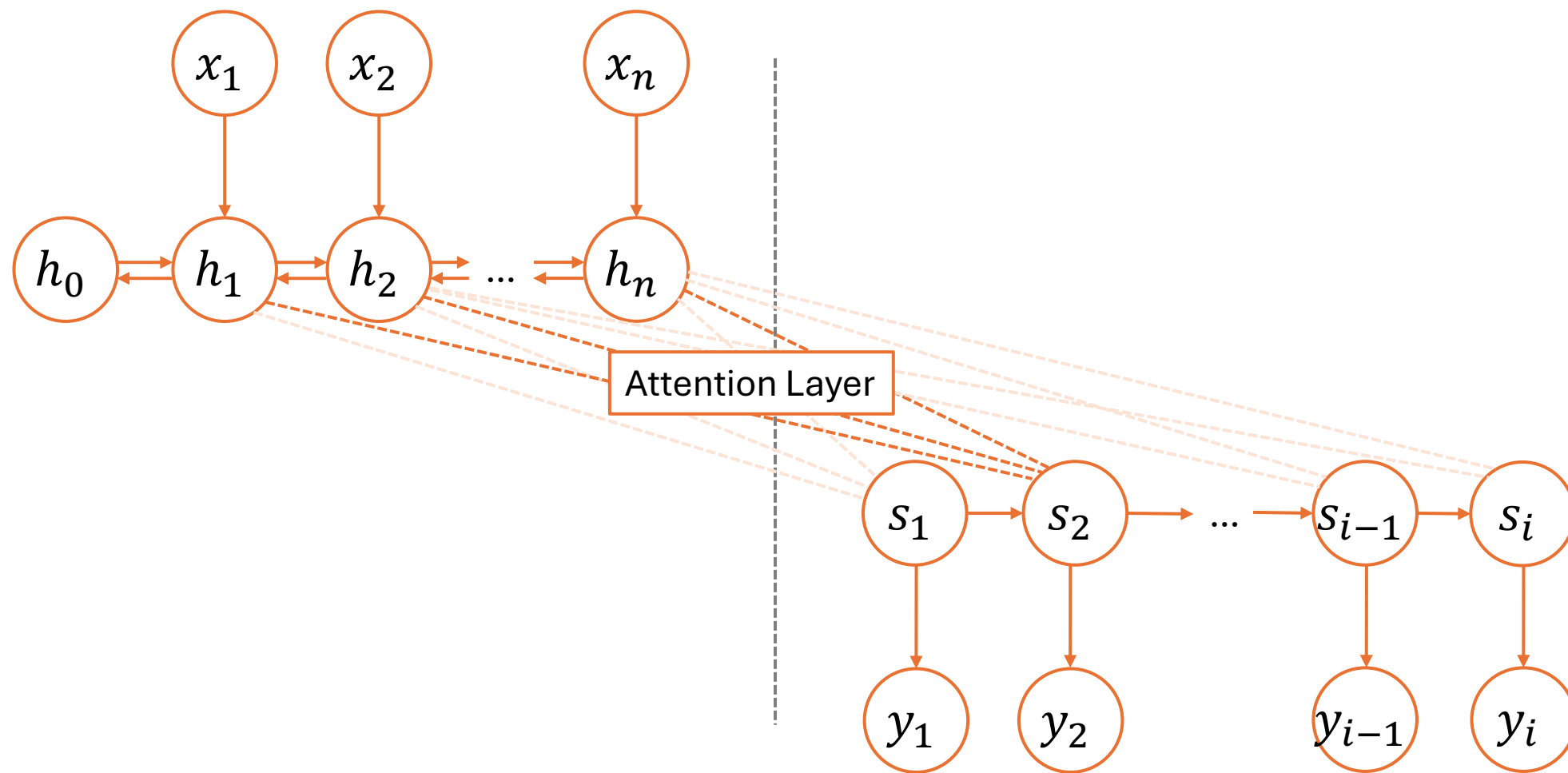
Modeling Language

Seq2Seq & Attention



Modeling Language

Seq2Seq & Attention



NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*

Université de Montréal

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*

Université de Montréal

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i | y_1, \dots, y_{i-1}, \mathbf{x}) = g(y_{i-1}, s_i, c_i), \quad (4)$$

where s_i is an RNN hidden state for time i , computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

It should be noted that unlike the existing encoder-decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i .

The context vector c_i depends on a sequence of *annotations* ($h_1, \dots, h_{T_x}$) to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i -th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_i :

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \quad (5)$$

The weight α_{ij} of each annotation h_j is computed by

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad (6)$$

where

$$e_{ij} = a(s_{i-1}, h_j)$$

is an *alignment model* which scores how well the inputs around position j and the output at position i match. The score is based on the RNN hidden state s_{i-1} (just before emitting y_i , Eq. (4)) and the j -th annotation h_j of the input sentence.

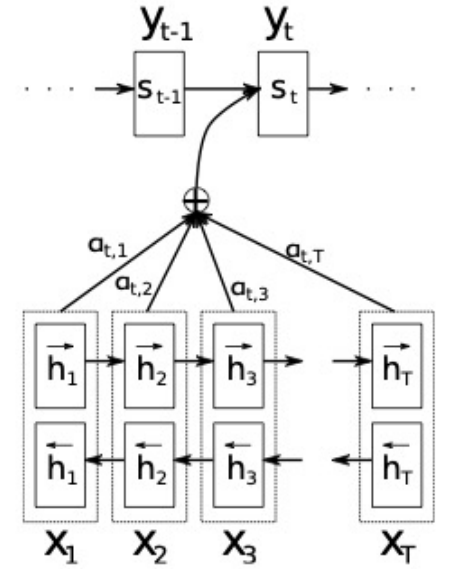


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Modeling Language

Seq2Seq & Attention

Problem Definition: “**Sequence-to-Sequence Translation**” (Next token prediction)

Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$

What problem can be phrased as “**Translation**” in Program Synthesis?

- Natural language to code generation
- Docstring $\rightarrow$ Implementation
- Code Repair: Buggy $\rightarrow$ Fixed Code
- Transpilation: Python Code $\rightarrow$ C Code

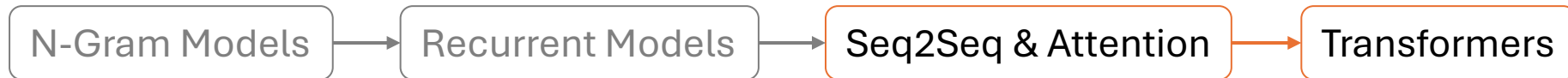
Modeling Language

Problem Definition: “**Sequence-to-Sequence Translation**” (Next token prediction)

Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$



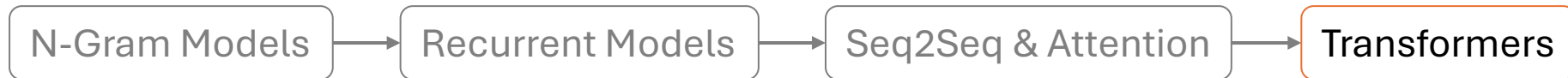
Modeling Language

Problem Definition: **Next token prediction**

Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$



Modeling Language

Transformers

Problem Definition: **Next token prediction**

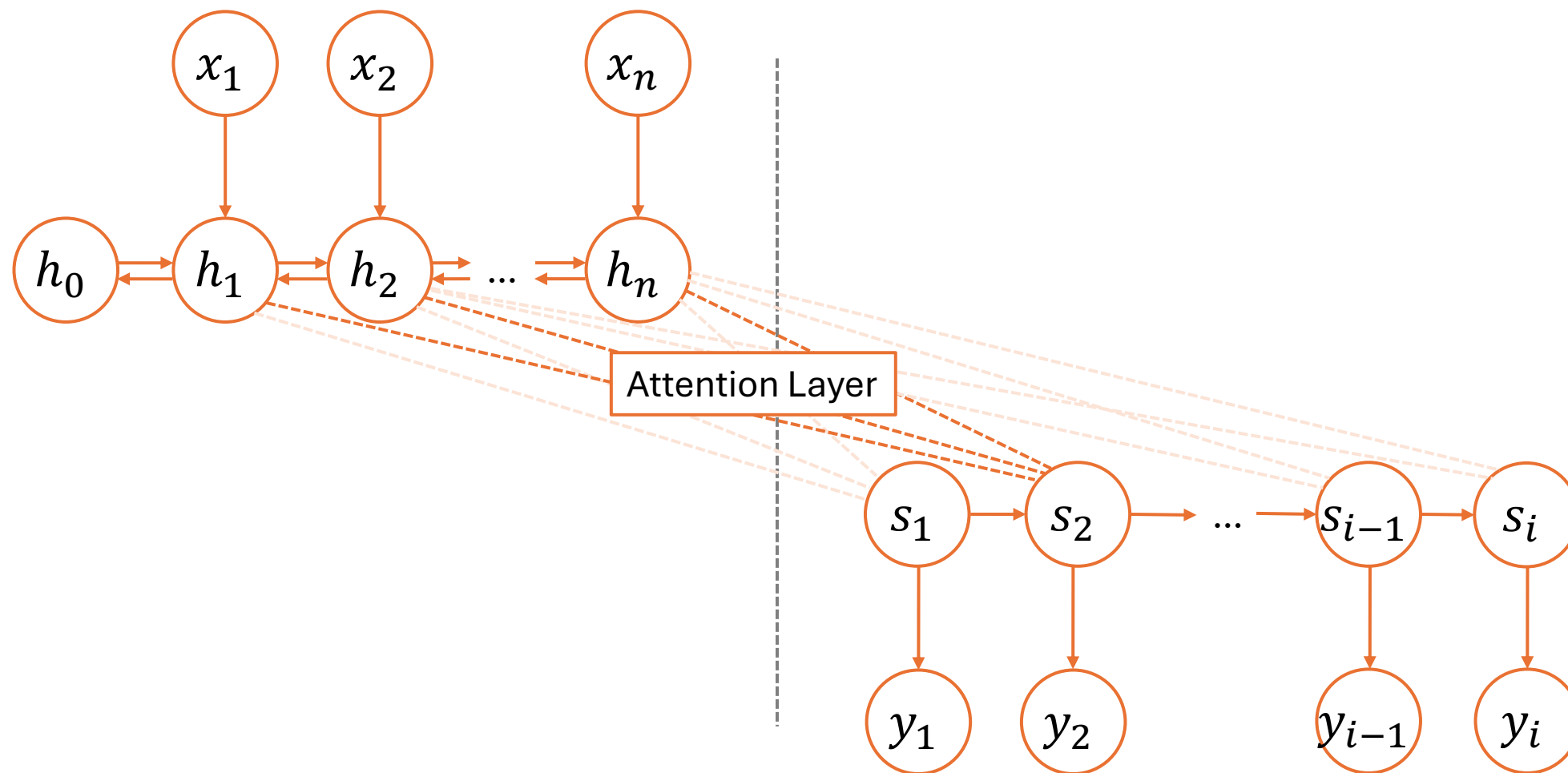
Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$

Modeling Language

Seq2Seq & Attention



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

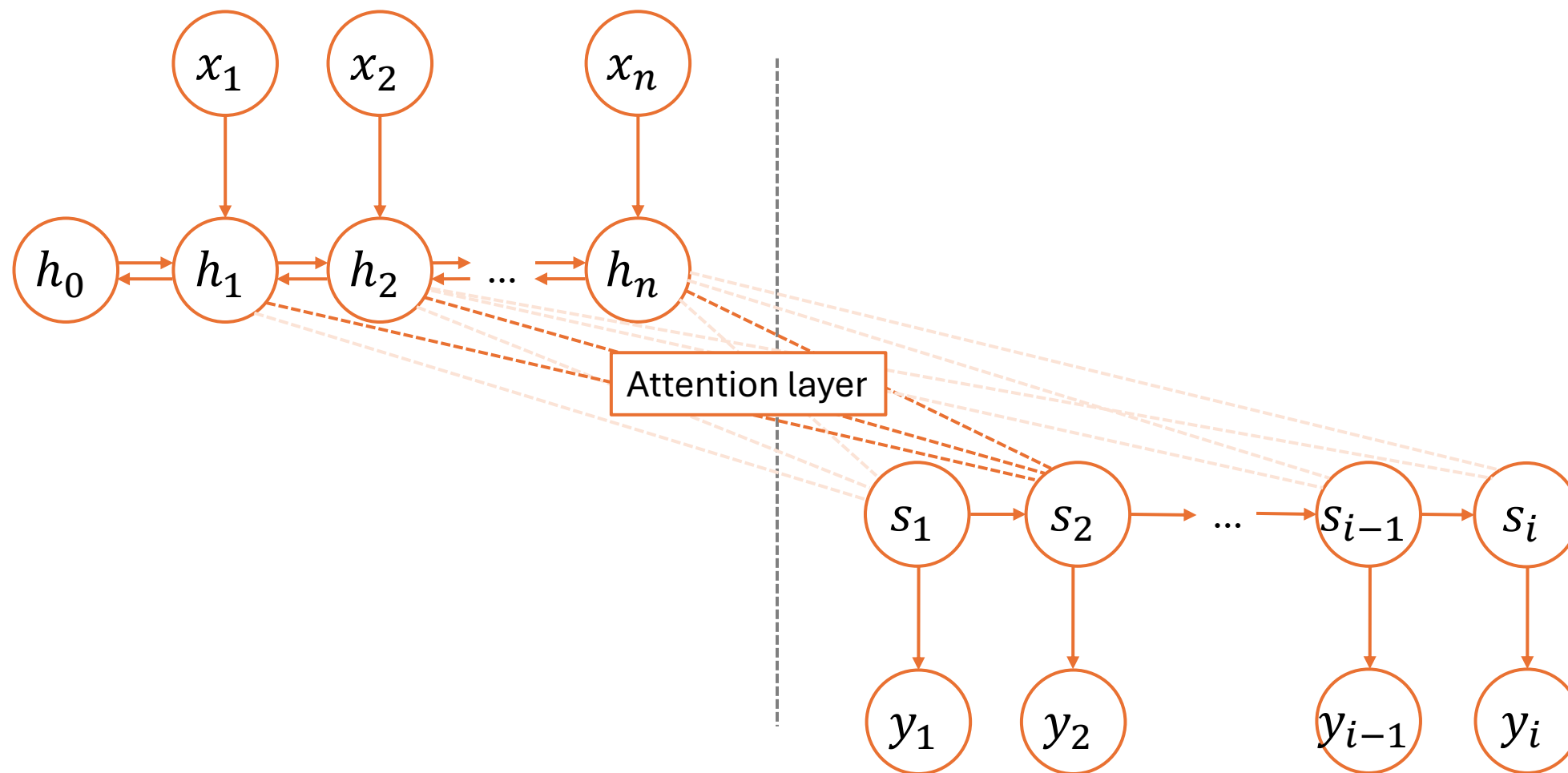
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

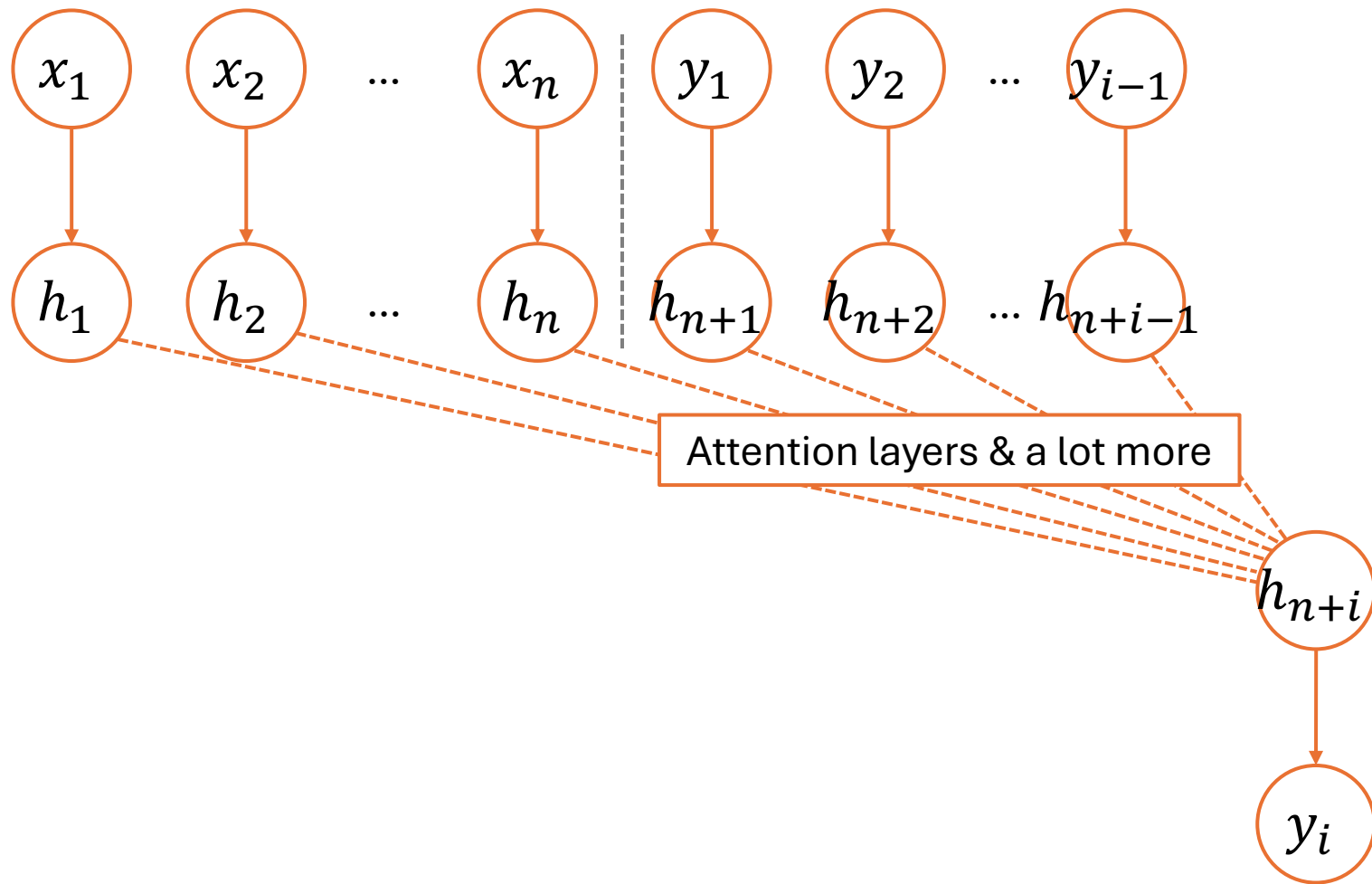
Modeling Language

Seq2Seq & Attention



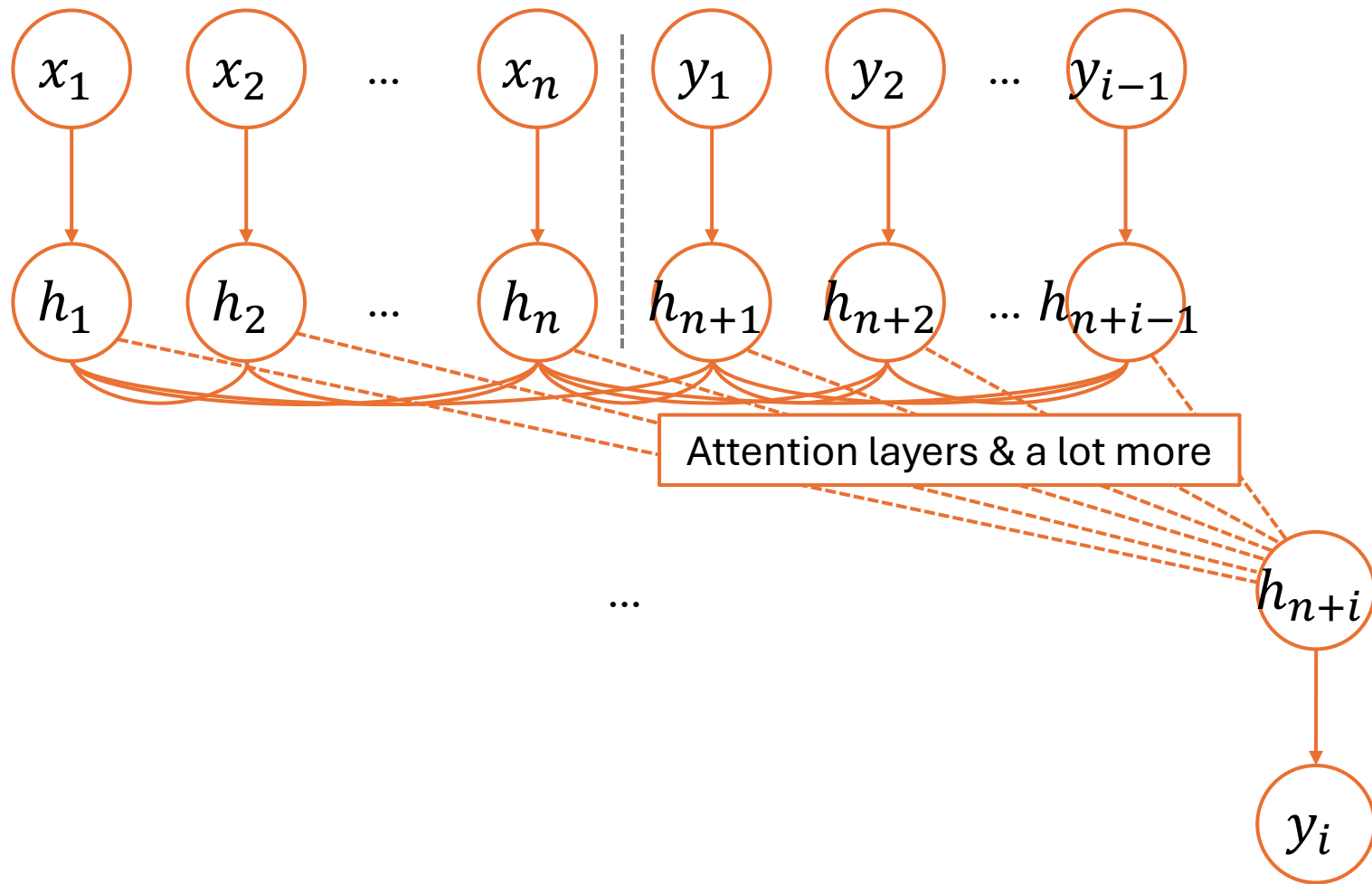
Modeling Language

Transformer



Modeling Language

Transformer



TRANSFORMER EXPLAINER

Examples ▾

for i in range(10): print(i)

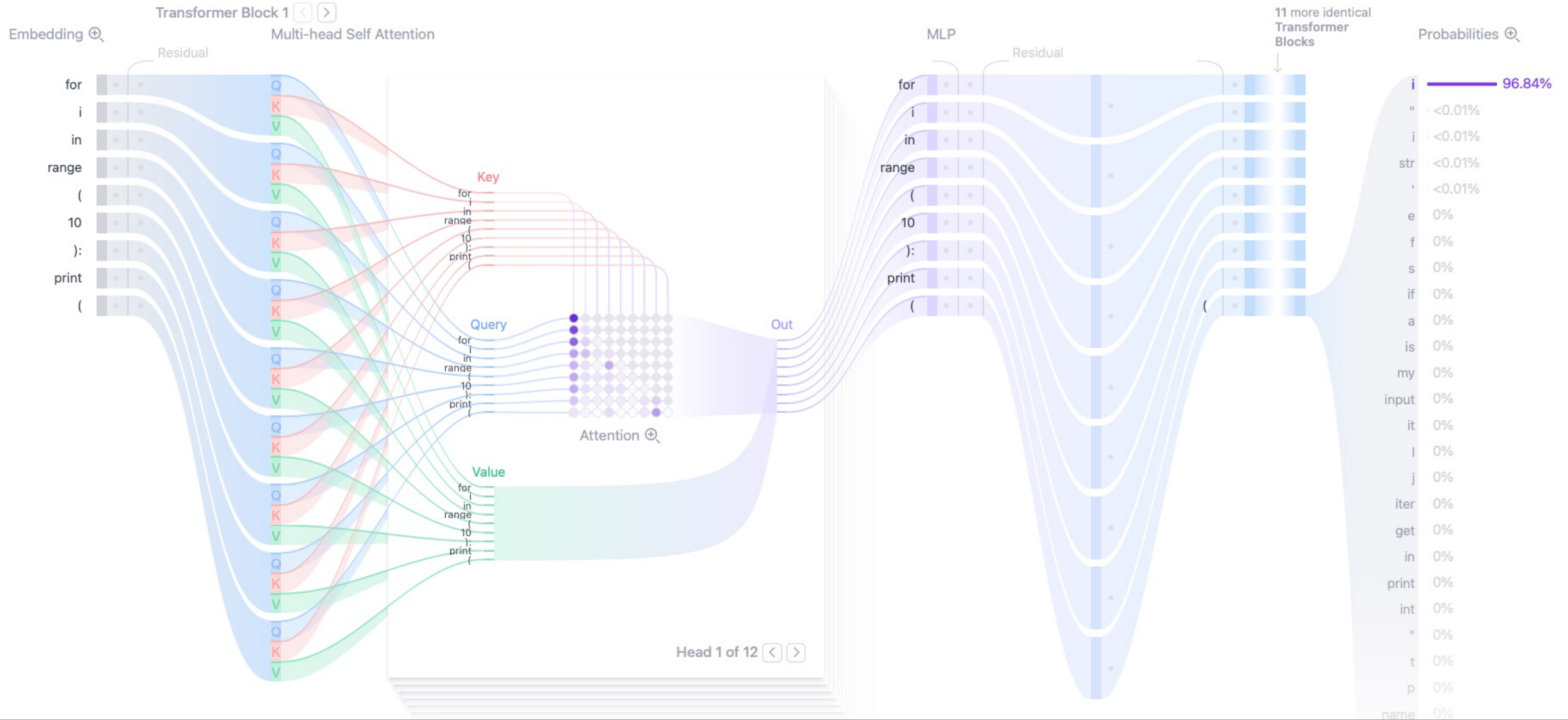
Generate

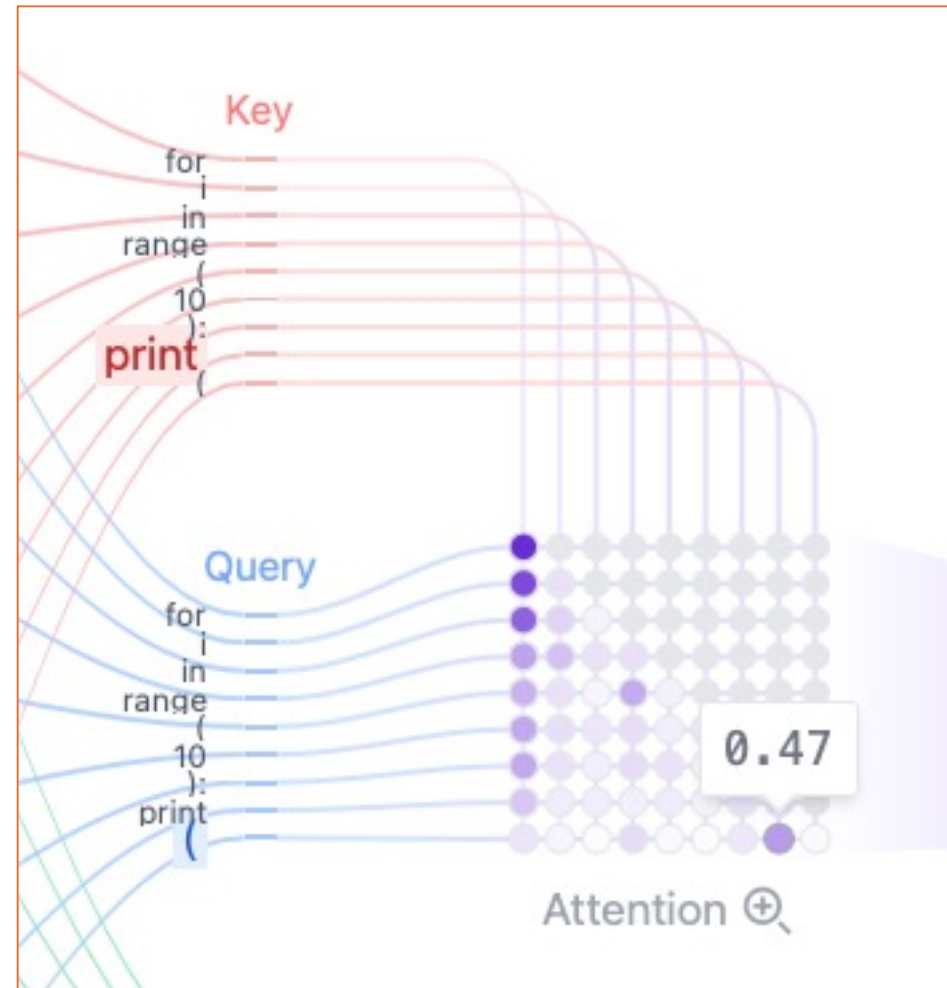
Temperature

0.8

Sampling ☒ Top-k ☐ Top-p

k=5





Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

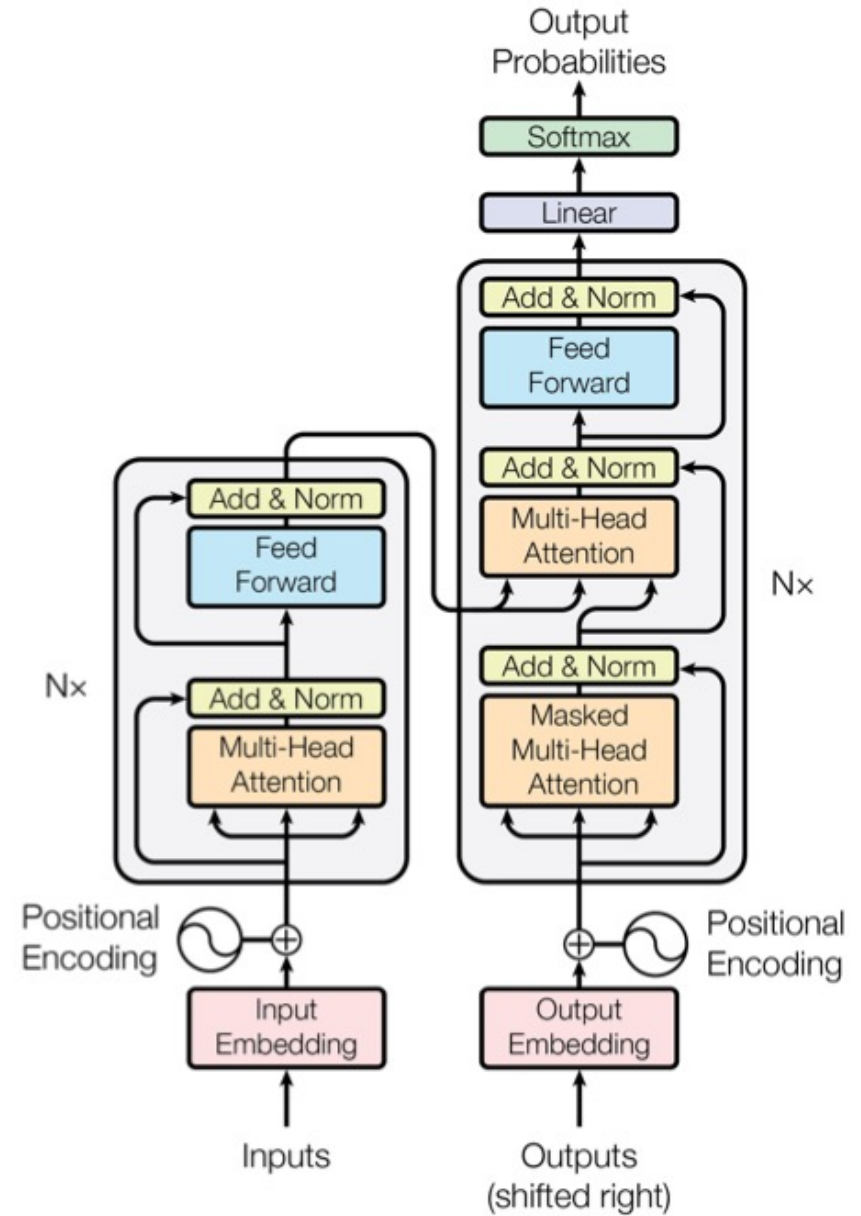


Figure 1: The Transformer - model architecture.

Modeling Language

Transformers

Problem Definition: **Next Token Prediction**

Input: **Input Sequence** $\mathbf{x} = x_1, x_2, \dots, x_n$ Output: **Next Token** y_i

Input: **Generated tokens so far** $y_1, y_2, \dots, y_{i-1}$

Goal: Compute the probability $\Pr(y_i \mid y_1, y_2, \dots, y_{i-1}, \mathbf{x})$

Transformers: How they work

- Self-Attention for the input
- Self-Attention for the output
- Cross-Attention for combining inputs and current output together
- Additional machinery: positional encodings, multi-headed attention, etc.
- Causal decoder-only models

Modeling Language

Problem Definition: **Next Token Prediction**

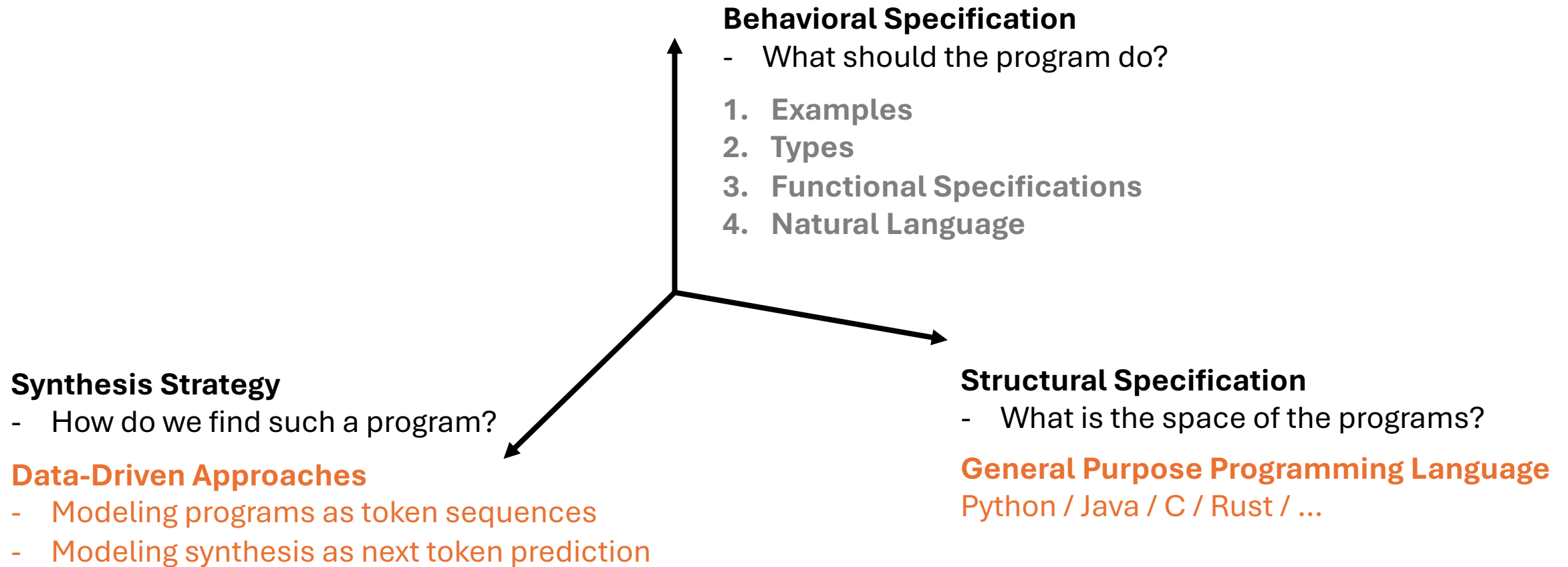
Input: **Prefix** $t_1, t_2, \dots, t_{i-1}$

Output: **Next Token** t_i

Goal: Compute the probability $\Pr(t_i \mid t_1, t_2, \dots, t_{i-1})$



Today



Week 3

- Assignment 1
 - <https://github.com/machine-programming/assignment-1>
 - Start working on the assignment early!
- Assignment 2
 - Will be released later this week!
- Waitlist
 - All waitlisted students should be able to enroll
 - Let me know if you do not have access to courselore and GradeScope
 - API keys should have been sent out